

# Sisteme de Operare - Întrebări de Examen

265 întrebări extrase din deck-ul Anki

---

## Întrebarea 1

Q:

Dați trei expresii regulate care se potrivesc cu orice număr nenegativ multiplu de 5.

A:

"\<[0-9]\*[05]\>"

"\<[1-9]\*0\*[05]\>"

---

## Întrebarea 2

Q:

Dati cinci comenzi GREP care afiseaza toate liniile dintr-un fisier care contin litera "a" mare sau mic.

A:

grep "[Aa]" file

grep -E "[Aa]" file

grep -i "a" file

grep -E "A|a" file

grep -iw "a" file

Observații:

- [Aa] = clasă de caractere care matchează A sau a (cea mai simplă)
  - -i = case insensitive (ignoră majuscule/minuscule)
  - A|a cu -E (extended regex) folosește | ca alternativă
  - -w cere ca match-ul să fie cuvânt întreg
- 

## Întrebarea 3

Q:

Scrieti doua comenzi SED care afiseaza dintr-un fisier doar liniile care nu contin cifra 7.

A:

```
sed "/7/d" file  
sed -n "/7!/p" file
```

Explicație:

- /7/d = șterge liniile care conțin 7
  - /7!/p cu -n = afișează doar liniile care NU conțin 7 (n = nu afișa implicit, ! = negație, p = print)
- 

## Întrebarea 4

Q:

Scrieti o comanda AWK care afiseaza suma penultimului camp al tutu

ror liniilor.

A:

```
awk '{ s += $(NF-1) } END { print s }' fisier
```

Explicație:

- NF = numărul de câmpuri pe linia curentă
  - \$(NF-1) = penultimul câmp
  - Adunăm suma în s pentru fiecare linie
  - END {} se execută o singură dată la final, afișează suma
- 

## Întrebarea 5

Q:

Cum puteti redirecta in linia de comanda iesirea de eroare prin pi  
pe inspre un alt program?

A:

```
command 2>&1 | other_program
```

Explicație: "2>&1" redirectează stderr (descriptor 2) la stdout (descriptor 1), iar apoi pipe-ul preia  
stdout-ul combinat.

Sau, dacă vrem doar stderr:

```
command 2>&1 >/dev/null | other_program
```

---

## Întrebarea 6

Q:

Scrieti un script Shell UNIX care afiseazd toate argumentele din 1

inia de comanda fara a folosi FOR.

**A:**

```
#!/bin/bash

while [ $# -gt 0 ]; do
    echo $0
    shift 1
done
```

---

## Întrebarea 7

**Q:**

Desenati ierarhia proceselor create de codul e mai jos, incluzand procesul parinte.

```
for(i=0; i<3; it++) {
    fork();
    execlp("ls", "ls", "/", NULL);
}
```

**A:**

Dacă execlp REUȘEȘTE: ierarhia este simplă  
 $P \rightarrow C1$

La  $i=0$ : P face  $\text{fork} \rightarrow C1$  (acum sunt 2 procese, P și C1).  
Apoi atât P cât și C1 ajung la  $\text{execlp}("ls", \dots)$ . Dacă execlp reușește, AMBELE procese sunt înlocuite cu programul "ls" și nu mai continuă bucla for. Astfel se creează DOAR 1 copil (C1).

Dacă execlp EȘUEAZĂ: codul continuă bucla, fiecare proces face fork la fiecare iterație  $\rightarrow 2^3 - 1 = 7$  copii.

---

## Întrebarea 8

**Q:**

Adăugați codul C necesar pentru ca fișierul b.txt să fie suprascris cu conținutul fișierului a.txt din instrucțiunea de mai jos.

```
execlp("cat", "cat", "a.txt", NULL);
```

**A:**

```
#include <fcntl.h>
#include <unistd.h>

int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO); // sau dup2(fd, 1);
close(fd);
execlp("cat", "cat", "a.txt", NULL);
```

Explicație:

- `open()` deschide `b.txt` pentru scriere (`O_WRONLY`), îl creează dacă nu există (`O_CREAT`) și îl golește (`O_TRUNC`)
  - `dup2(fd, STDOUT_FILENO)` redirecționează `stdout` (descriptor 1) la fișierul `b.txt`
  - după `execvp`, "cat" va scrie conținutul lui `a.txt` pe `stdout`, care e acum `b.txt`
  - rezultat: `b.txt` va conține tot ce era în `a.txt`
- 

## Întrebarea 9

Q:

De ce nu e recomandat sa comunicati bidirectional printr-un singur FIFO?

A:

Răspuns:

Nu suntem siguri de ordinea în care procesele vor scrie în FIFO, ceea ce poate cauza coruperea datelor.

Să presupunem că procesul A vrea să scrie "abc" și apoi procesul B "123".

Dacă A pierde CPU-ul în timpul scrierii, putem avea ceva de genul:  
a1bc23

---

## Întrebarea 10

Q:

Cate FIFO-uri poate deschide un process daca nu sunt și nici nu vo

r fi folosite vreodata de vreun alt proces?

A:

Răspuns:

Un proces poate deschide cel mult un FIFO dacă nu există alt proces care să folosească acel FIFO, deoarece funcția `open` așteaptă până când un alt proces deschide FIFO-ul pentru capătul complementar.

---

## Întrebarea 11

Q:

Cand ati folosi un process in locul unui thread?

A:

Folosesc procese în loc de thread-uri când:

- Vreau izolare de memorie (un crash într-un proces nu afectează celelalte)
- Programele trebuie să fie complet independente (alte executabile via exec)
- Vreau să rulez pe mașini diferite (procesele pot fi distribuite, thread-urile nu)
- Lucrez cu cod care nu e thread-safe (biblioteci vechi)
- Securitate: procesele au granițe clare de securitate (spații de adrese diferite)
- Vreau să rulez programe externe (ls, grep, etc.) — necesită fork + exec

---

## Întrebarea 12

Q:

Ce este o "secțiune critică"?

A:

Răspuns:

O secțiune critică este o porțiune de cod executată de mai multe thread-uri și care conține o resursă critică (mai multe thread-uri pot modifica acea resursă).

---

## Întrebarea 13

Q:

De ce un thread trebuie să verifice condiția la ieșirea din apelul `pthread_cond_wait`?

`pthread_cond_wait` call?

A:

Răspuns:

Un thread ar trebui să verifice condiția după întoarcerea din `pthread_cond_wait` pentru că și alte thread-uri pot aștepta pe aceleași variabile condiționale și pot fi notificate înaintea thread-ului curent, ceea ce ar putea face ca resursa cerută să nu mai fie disponibilă în cantitatea necesară.

---

## Întrebarea 14

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `pthread_rwlock_rdlock`?

`pthread_mutex_lock` with calls to `pthread_rwlock_rdlock`?

**A:**

Răspuns:

Dacă înlocuim `pthread_mutex_lock` cu `pthread_rwlock_rdlock`, nu vom mai proteja resursa critică, deoarece `rdlock` permite mai multor cititori în același timp, dar nu este sigur că acești cititori nu vor modifica resursa.

Dacă resursa nu este modificată de cititori (doar citirea este necesară de către thread-uri), atunci înlocuirea va duce la o performanță mai bună.

---

## Întrebarea 15

**Q:**

Care e efectul apelului la `pthread_barrier_wait` pentru o bariera inițializată cu 1?

`pthread_barrier_wait` on a barrier initialized with 1?

**A:**

Răspuns:

Nu există niciun efect la așteptarea pe o barieră cu valoare inițială 1, deoarece thread-ul care apelează `wait` va face bariera să se decrementeze la 0 și va permite thread-ului să treacă, resetând bariera înapoi la 1.

---

## Întrebarea 16

**Q:**

Cum puteți incrementa valoarea unui semafor?

**A:**

Răspuns:

Pentru a incrementa valoarea unui semafor putem folosi funcția `sem_post`.

---

## Întrebarea 17

**Q:**

Ce puteți face ca programator pentru a preveni deadlock-urile?

**A:**

one simple solution for avoiding deadlocks is to choose an order in which we lock the resources and stick to that order.

---

## Întrebarea 18

Q:

Prin ce tranziție de stare va trece un process când citește dintr-un fișier?

A:

Răspuns:

Procesul va trece în starea de așteptare (WAIT) până când conținutul fișierului cerut este încărcat în memorie de pe disc.

---

## Întrebarea 19

Q:

Ce conține superblocul unui disc Linux?

A:

Răspuns:

Superblocul de pe un disc Linux conține metadate despre organizarea sistemului de fișiere. Cunoaște unde începe fiecare partiție, dimensiunea discului etc.

---

## Întrebarea 20

Q:

Se poate crea un link hard spre un fișier aflat pe o altă partiție?  
? Justificați răspunsul.

A:

Răspuns:

Nu putem avea un hard link către un fișier de pe altă partiție, deoarece un hard link ține evidența i-node-ului fișierului. Din moment ce i-node-urile sunt disponibile doar în interiorul aceleiași partiții, nu putem avea un hard link către un fișier de pe altă partiție pentru că nu avem acces la numărul i-node-ului.

---

## Întrebarea 21

Q:

Dați o expresie regulată care acceptă orice secvență de lungime pară de cuvinte (formate din litere) separate prin spații, cu condiția ca pentru fiecare cuvânt să lungimea și poziția în secvență să fie ambele pare sau impare. Numărarea cuvintelor începe de la 1. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea să aibă lungime pară.

**A:**

```
^((([a-z][a-z])*[a-z]( )+([a-z][a-z])+( )+)*)*$
```

ANSWER 2:

```
^((([a-z][a-z])*[a-z]( )+([a-z][a-z])+( )+)*)*$
```

---

## Întrebarea 22

**Q:**

Dați trei comenzi GREP care afișează dintr-un fișier liniile formate exclusiv dintr-o secvență nevidă de litere și cifre alternativ (ex: a0g sau 1r5m).

**A:**

Answer:

```
grep -E -i "^(^([0-9]?([a-z][0-9])*)$|^([a-z]?([0-9][a-z])*)$)"
```

```
grep -E "(^([0-9]?([a-zA-Z][0-9])*)$|^([a-zA-Z]?([0-9][a-zA-Z])*)$)"
```

```
grep -E -i -v "[a-z][a-z][0-9][0-9]" a.txt
```

```
grep -E "((^(\d?([a-zA-Z]\d)*$)|(^([a-zA-Z]?(\d[a-zA-Z])*$)))" file.txt
```

```
grep -E -i "((^(\d?([a-z]\d)*$)|(^([a-z]?(\d[a-z])*$)))" file.txt
```

```
grep -E "((^[0-9]?([a-zA-Z][0-9])*$)|(^([a-zA-Z]?([0-9][a-zA-Z])*$)))" file.txt
```

---

## Întrebarea 23

**Q:**

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență nevidă de litere mici.

**A:**

```
sed -E 's/([a-z]+)//'
```

```
sed -E 's/([a-z][a-z]*)/'
```

Answer:

```
sed -E 's/[a-z]+/'
```

```
sed 's/[a-z]\+/'
```



---

## Întrebarea 24

Q:

Scriveți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărei linii sunt formate din secvențe de cifre separate prin spații.

A:

```
awk 'BEGIN {sum = 0} {for (i = 1; i <= NF; i++) sum += $i} END { print sum}'
```

Answer:

```
awk "{for(i = 1; i <= NF; i++) SUM += $i} END {print SUM}"
```

---

## Întrebarea 25

Q:

Dați trei moduri de a afla dimensiunea unui fișier în linia de comandă Linux.

A:

Răspuns:

1) du

Exemplu: du fisier

2) ls -s

Exemplu: ls -s fisier

3) wc -c fisier

Alternative:

ls -lh fisier

du -h fisier

stat -c %d fisier

---

## Întrebarea 26

Q:

Scriveți o condiție Shell UNIX care verifică dacă un fișier există și utilizatorul curent are vreo permisiune asupra lui.

A:

Răspuns:

```
[ -e fisier ] && [ -r fisier -o -w fisier -o -x fisier ]
```

Sau echivalent:

```
[ -e fisier ] && { [ -r fisier ] || [ -w fisier ] || [ -x fisier ]; }
```

---

## Întrebarea 27

**Q:**

Desenati ierarhia proceselor create de codul de mai jos, incluzand procesul parinte.

```
for(i=0; i<3; i++) {  
    if(fork() > 0) {  
        wait(0);  
        wait(0);  
        exit(0);  
    }  
}exam11july2023.txt
```

**A:**

Ierarhia este un LANȚ:  $P \rightarrow C1 \rightarrow C2 \rightarrow C3$

Explicație pas cu pas:

-  $i=0$ : P face fork  $\rightarrow C1$ .

P ( $\text{fork} > 0$ ) intră în if și face `wait(0)`, `wait(0)`, `exit`. P se termină.

C1 ( $\text{fork} == 0$ ) NU intră în if, continuă bucla.

-  $i=1$ , în C1: fork  $\rightarrow C2$ .

C1 ( $\text{fork} > 0$ ) face `wait`, `wait`, `exit`.

C2 ( $\text{fork} == 0$ ) continuă bucla.

-  $i=2$ , în C2: fork  $\rightarrow C3$ .

C2 face `wait`, `wait`, `exit`.

C3 continuă bucla, iese ( $i$  devine 3) și se termină.

Observație: codul are un bug — `wait(0)` e apelat de două ori, dar fiecare proces are doar UN copil, nu doi. Al doilea `wait` va returna -1 (`errno = ECHILD`).

---

## Întrebarea 28

**Q:**

De ce descriptorul de fișier returnat de `popen` trebuie închis cu `pclose` și nu cu `fclose`?

**A:**

Răspuns:

La implementarea ei, `popen` creează un proces nou, după care trebuie să așteptăm. Dacă am folosi `fclose`, nu am aștepta procesul, deci trebuie să folosim `pclose`, deoarece aceasta așteaptă terminarea procesului și returnează statusul de ieșire al comenzii.

Deoarece popen deschide un nou proces care trebuie închis cu pclose. Pclose închide pointer-ul de fișier, așteaptă terminarea procesului asociat și returnează statusul terminării, în timp ce fclose ar închide doar stream-ul de fișier, fără a îndeplini celelalte sarcini.

---

## Întrebarea 29

Q:

Când ați folosi `execv` în locul de `execl`?

`execv` instead of `execl`?

exam11july2023

A:

Răspuns:

Aș folosi `execv` în loc de `execl` atunci când vreau să am argumentele comenzii de executat stocate într-o variabilă. De exemplu, ar fi util când vrem să folosim argumentele din linia de comandă care sunt stocate în `char **argv`.

Aș folosi `execv` când trebuie să transmit un array de argumente programului nou, util când numărul de argumente este stocat, permițând transmiterea argumentelor procesului nou.

---

## Întrebarea 30

Q:

Dați trei apeluri de funcții care asigură excludere mutuală.

A:

Răspuns:

- 1) `pthread_mutex_lock(&mtx)`
  - 2) Folosind semafoare binare: `sem_wait(&sem)`
  - 3) `pthread_rwlock_wrlock(&rwlock)`
- 

## Întrebarea 31

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_post`?

`pthread_mutex_lock` with calls to `sem_post`?

A:

Răspuns:

Haos. `sem_post` incrementează valoarea semaforului, deci nu va bloca niciun thread de la a intra într-o secțiune critică doar prin el însuși. Nu vom avea niciun mecanism de prevenire pe secțiunea critică, așa că putem accesa concurent locații de memorie, rezultând în răspunsuri nedorite.

Înlocuirea `pthread_mutex_lock` cu `sem_post` ar duce la haos, deoarece `sem_post` incrementează semaforul, permițând oricărui thread să intre în secțiunea critică, eșuând astfel în prevenirea accesului concurent la resursele partajate. Acest lucru ar putea duce la race conditions și corupere de date. Mai mult, dacă înlocuim doar `pthread_mutex_lock` dar păstrăm `pthread_mutex_unlock`, va rezulta într-un comportament nedefinit, deoarece deblocăm un mutex care nu a fost niciodată blocat.

---

## Întrebarea 32

**Q:**

Ce se poate întâmpla dacă funcția `f` este rulată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    ...
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

**A:**

Răspuns:

Ar putea duce la deadlock. Dacă un thread are `id%2 = 0` și celălalt `id%2 = 1`, și încearcă să blocheze simultan, primul ar bloca `m[0]`, celălalt ar bloca `m[1]`, iar amândoi vor aștepta pentru `m[1]`, respectiv `m[0]` să se elibereze, simultan. Deci avem un deadlock.

---

## Întrebarea 33

**Q:**

Dați un exemplu de valori distincte și mai mari ca 0 pentru `T`, `N1`, `N2` și `N3` pentru care programul de mai jos se încheie.

```
pthread_barrier_t b1, b2;

void* f1(void* a) {
    pthread_barrier_wait(&b1);
    return NULL;

void* f2(void* a) {
    pthread_barrier_wait(&b2);
    return NULL;
```

```

int main() {
    int i;
    pthread_t t[T][2];

    pthread_barrier_init(&b1, NULL, N1);

    pthread_barrier_init(&b2, NULL, N2);

    for(i=0; i<T; i++) {
        pthread_create(&t[i][0], NULL, f1, NULL);
        pthread_create(&t[i][1], NULL, f2, NULL);

        i

        for(i=0; i<T; i++) {
            pthread_join(t[i][0], NULL);
            pthread_join(t[i][1], NULL);

            y

        pthread_barrier_destroy(&b1);

        pthread_barrier_destroy(&b2);

        return NULL;
    }
}

```

**A:**

$T = 4, N1 = 2, N2 = 2$  (sau orice valori unde  $T$  este multiplu atât al lui  $N1$  cât și al lui  $N2$ )

Explicație:

- $T$  thread-uri sunt create care apelează  $f1$  (deci  $T$  thread-uri ajung la bariera  $b1$ )
- $T$  thread-uri apelează  $f2$  ( $T$  ajung la bariera  $b2$ )
- Bariera  $b1$  se eliberează când  $N1$  thread-uri ajung la ea
- Pentru ca toate cele  $T$  thread-uri să poată trece,  $T$  trebuie să fie multiplu de  $N1$
- La fel pentru  $b2$ :  $T$  trebuie să fie multiplu de  $N2$

Deci condiția:  $T \% N1 == 0$  ȘI  $T \% N2 == 0$ .

Valori distincte și  $> 0$ :  $T=6, N1=2, N2=3$ . Sau  $T=4, N1=2, N2=4$ . Sau  $T=12, N1=3, N2=4$ .

## Întrebarea 34

**Q:**

Ce puteți face ca programator pentru a preveni deadlock-urile?

**A:**

Deadlock-urile pot fi prevenite împiedicând cel puțin una dintre cele patru condiții necesare:

1. Eliminarea așteptării circulare: alegeți o ordine pentru resurse și blocați-le mereu în aceeași ordine.

2. Eliminarea excluderii mutuale:

- Resursele partajate cum ar fi fișierele read-only nu duc la deadlock, dar nu este posibil să eliminăm complet excluderea mutuală, deoarece unele resurse (cum ar fi imprimanta) sunt inherent non-partajabile.

3. Eliminarea condiției Hold and Wait:

- Alocați toate resursele necesare procesului înainte de începerea execuției. Asta elimină condiția hold-and-wait dar duce la utilizare scăzută a dispozitivelor.
- Procesul cere resurse noi doar după ce le eliberează pe cele curente. Această soluție poate duce la starvation.

4. Eliminarea așteptării circulare:

- O modalitate este numerotarea tuturor resurselor și cerința ca procesele să le ceară doar în ordine strict crescătoare (sau descrescătoare).

5. Eliminarea condiției No Preemption:

- Preemptiunea alocării resurselor poate preveni această condiție când e posibil.

O metodă concretă pentru prevenirea deadlock-urilor este stabilirea unei ordini de blocare și respectarea ei mereu pentru a evita ciclurile. De exemplu, în problema filozofilor, putem bloca întotdeauna mai întâi furculița cu indicele mai mic, apoi pe cea cu indicele mai mare.

---

## Întrebarea 35

**Q:**

Prin ce tranziție de stare va trece un process când apelează `sem_wait` și în ce condiții? Justificați răspunsul.

**A:**

Răspuns:

Dacă valoarea semaforului este 0, thread-ul va fi pus în WAIT până când un slot devine disponibil, pentru că altfel ar irosi un core fără să facă nimic.

Dacă valoarea este diferită de 0, atunci `sem_wait` nu va avea niciun efect asupra thread-ului respectiv, deci va rămâne probabil în RUN, dacă nu este comandat de alți factori externi să treacă în READY.

- dacă valoarea semaforului  $> 0$ , procesul decrementează semaforul și continuă execuția în starea RUNNING

- dacă semaforul = 0, procesul trece în starea WAITING (blocat) până când semaforul este incrementat prin `sem_post`.

---

## Întrebarea 36

Q:

Considerand ca dimensiunea unui bloc este B si dimensiunea unei adrese este A, cate blocuri de date sunt adresate de indirectarea dubla a unui i-nod?

A:

$(B/A)^2$  blocuri de date.

Explicație:

- Un bloc are dimensiunea B octeți
- O adresă (pointer la bloc) are dimensiunea A octeți
- Deci un bloc poate conține  $B/A$  adrese
- Indirectarea dublă: pointer → bloc cu  $B/A$  adrese → fiecare adresă spre un bloc cu  $B/A$  adrese → fiecare spre un bloc de date
- Total blocuri de date:  $(B/A) \times (B/A) = (B/A)^2$

---

## Întrebarea 37

Q:

Cand ati incarca in memorie paginile unui program care tocmai este pornit?

A:

Răspuns:

Aș încărca o pagină în memorie atunci când este cerută, deoarece dacă le încarc pe toate odată, efectul de paginare ar dispărea.

La pornire, aș încărca prima pagină, preîncărcând vecinii, bazat pe principiul localității. Acesta spune că un proces va avea nevoie în curând de paginile vecine paginii tocmai încărcate. După aceea, aș lua unul dintre vecini, repetând principiul.

---

## Întrebarea 38

Q:

Dați trei expresii regulate care acceptă orice număr ne-negativ multiplu de 5.

A:

1.  $^[0-9]^*[05]^\$$
2.  $^0^\$|^[0-9]^*[05]^\$$
3.  $^5^\$|^[0-9]^*[05]^\$$

Cred ca ar mai merge si:  $^\<[0-9]^*[05]^\>$  (nu stiu sigur daca trb sa fie toata linia sau doar numarul)

---

## Întrebarea 39

Q:

Dați cinci comenzi GREP care afișează toate liniile dintr-un fișier care conțin litera "a" mare sau mic.

A:

```
grep -i "a" filename  
grep -Ei "a" filename  
grep "[aA]" filename  
grep -i "[a]+" filename  
grep -iw "a" filename
```

Observație: Pentru a treia, trebuie să fie `grep -E -i "[a]+" filename`. Fără `-E` încearcă să caute literalmente cuvântul `[a]+` și nu-l găsește.

---

## Întrebarea 40

Q:

Scrieți două comenzi SED care afișează dintr-un fișier doar liniile care nu conțin cifra 7.

A:

```
sed -E "/7/d" filename  
sed -n "/7!/p" filename
```

Mie ultima nu mi merge in terminal cand o scriu(imid da ceva eroare).

Poate se pune ca o comanda diferita asta: `sed -E "/[7]+/d" filename`

Saaaaau mai merge si asa: `sed -E "s/^(.*[7].*)$/gi" filename` (cu toate ca aici liniile care au 7 doar le face goale)

---

## Întrebarea 41

Q:

Scrieți o comandă AWK care afișează suma penultimului câmp al tuturor liniilor.

A:

```
awk "{SUM += $(NF-1)} END{print SUM}" filename
```

---

## Întrebarea 42

Q:



Cum puteți redirecta în linia de comanda ieșirea de eroare prin pipe înspre un alt program?

**A:**

Răspuns:

command 2>&1 | another\_program

Sau:

- Redirecționează erorile unei comenzi într-un fișier: `rm fisier-inexistent.c 2> output.err`
- Redirecționează atât stdout cât și stderr în același fișier: `rm fisier-inexistent.c > output.all 2>&1` - citiți ca "redirecționează stdout în output.all, iar stderr în același loc unde merge stdout".

---

## Întrebarea 43

**Q:**

Scrieți un script Shell UNIX care afișează toate argumentele din linia de comandă fără a folosi FOR.

**A:**

```
#!/bin/bash
echo "$@"
```

Sau, dacă vrem fiecare argument pe o linie separată:

```
#!/bin/bash
while [ $# -gt 0 ]; do
    echo "$1"
    shift
done
```

Explicație:

- "\$@" expandează la toate argumentele
- shift mută argumentele la stânga (\$2 devine \$1, \$3 devine \$2, etc.)
- \$# este numărul de argumente rămase

---

## Întrebarea 44

**Q:**

Desenați ierarhia proceselor create de codul de mai jos, incluzând procesul părinte.

```
for(i=0; i<3; i++) {
    fork();
    execlp("ls", "ls", "/", NULL);
}
```

**A:**

Răspuns:

P creează 3 copii (C1, C2, C3). Fiecare proces (inclusiv P) execută `execlp("ls", "ls", "/", NULL)`. Dacă `execlp` reușește, procesul este înlocuit cu "ls" și nu se mai întoarce în buclă.

Ierarhia:

```
P -> C1
    -> C2
    -> C3
```

În total se creează 3 procese copil (sau 7, dacă socotim toate creațiile dacă `execlp` ar eșua și bucla ar continua).

---

## Întrebarea 45

Q:

Adăugați codul C necesar pentru ca fișierul `b.txt` să fie suprascris cu conținutul fișierului `a.txt` din instrucțiunea de mai jos.

```
execlp("cat", "cat", "a.txt", NULL);
```

A:

```
#include <fcntl.h>
#include <unistd.h>

int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO);
close(fd);
execlp("cat", "cat", "a.txt", NULL);
```

Explicație:

- `open()` deschide `b.txt` cu `O_WRONLY` (scriere), `O_CREAT` (creează dacă nu există) și `O_TRUNC` (îl golește dacă există)
- `dup2(fd, STDOUT_FILENO)` face ca tot ce scrie procesul pe `stdout` să ajungă în `b.txt`
- după `execlp`, `cat` scrie `a.txt` pe `stdout` → ajunge în `b.txt`
- rezultat: `b.txt` va conține conținutul lui `a.txt`

---

## Întrebarea 46

Q:

De ce nu e recomandat să comunicați bidirecțional printr-un singur FIFO?

A:

Trebuie să creăm 2 FIFO-uri pentru comunicarea bidirecțională, deoarece un FIFO poate transmite date doar într-o singură direcție și astfel evităm comportament neașteptat al programului.

Folosirea unui singur FIFO poate duce la race conditions și deadlock-uri.

În plus: nu suntem siguri de ordinea în care procesele vor scrie în FIFO, ceea ce poate cauza corupere de date. Să presupunem că procesul A vrea să scrie "abc" și apoi procesul B "123". Dacă A pierde CPU-ul în timpul scrierii, putem avea ceva de genul "a1bc23".

---

## Întrebarea 47

**Q:**

Câte FIFO-uri poate deschide un process dacă nu sunt și nici nu vor fi folosite vreodată de vreun alt proces?

**A:**

Niciuna care să nu se blocheze.

Dacă procesul deschide un FIFO pentru SCRIERE și niciun alt proces nu îl deschide pentru CITIRE → `open()` se blochează la nesfârșit.

Dacă procesul deschide pentru CITIRE și nimeni nu îl deschide pentru SCRIERE → `open()` se blochează.

Dacă FIFO-ul este deschis cu `O_RDWR` (citire+scriere) de către același proces, atunci `open()` nu se blochează, dar deschiderea e considerată "self-pipe" și nu prea are sens.

Teoretic, numărul maxim de FIFO-uri ar fi limitat de `ulimit -n` (file descriptor limit).

---

## Întrebarea 48

**Q:**

Când ați folosi un process în locul unui thread?

**A:**

- Procesele au propriul spațiu de memorie, oferind izolare și reducând coruperea memoriei.
  - Procesele pot fi distribuite pe mai multe mașini, în timp ce thread-urile sunt limitate la resursele unei singure mașini.
  - Procesele oferă granițe de securitate mai bune, deoarece nu partajează același spațiu de adrese.
  - O eroare într-un proces nu afectează celelalte procese.
- 

## Întrebarea 49

**Q:**

De ce un thread trebuie să reverifice condiția la ieșirea din apelul `pthread_cond_wait`?

pthread\_cond\_wait call?

**A:**

Un thread trebuie să reverifice condiția după pthread\_cond\_wait pentru că:

1. Spurious wakeups (treziri spontane): sistemul poate trezi un thread fără un semnal explicit (e o caracteristică POSIX permisă).
2. Race conditions cu alte thread-uri: după ce un thread e trezit prin signal/broadcast, există un decalaj de timp până când reacheziționează mutex-ul. Alt thread care aștepta pe aceeași condiție poate fi trezit primul, achiziționa mutex-ul și consuma resursa, modificând astfel condiția.

Soluție: folosim mereu while (nu if) pentru verificare:

```
while (!condiție_îndeplinită)
    pthread_cond_wait(&c, &m);
```

---

## Întrebarea 50

**Q:**

Care va fi efectul înlocuirii apelurilor la pthread\_mutex\_lock cu apeluri la pthread\_rwlock\_rdlock?

pthread\_mutex\_lock with calls to pthread\_rwlock\_rdlock?

**A:**

Va cauza probleme deoarece pthread\_rwlock\_rdlock este folosit pentru blocaje de citire, permițând mai multor thread-uri să acceseze secțiunea critică, ceea ce nu asigură excluderea mutuală și ar putea duce la race conditions.

---

## Întrebarea 51

**Q:**

Care e efectul apelului la pthread\_barrier\_wait pentru o barieră inițializată cu 1?

**A:**

Va elibera imediat thread-ul care așteaptă, deoarece condiția barierei este deja satisfăcută. Nu va avea nevoie de nicio așteptare, iar thread-ul va continua.

(Va funcționa similar cu un pthread\_mutex.)

---

## Întrebarea 52

**Q:**

Cum puteți incrementa valoarea unui semafor?

**A:**

Folosind funcția `sem_post()`:

```
#include <semaphore.h>
sem_post(&sem);
```

`sem_post` incrementează valoarea semaforului cu 1. Dacă există thread-uri care așteaptă pe `sem_wait`, unul dintre ele este trezit.

Este operația V (sau "up", sau "signal") din terminologia clasică a semaforilor.

---

## Întrebarea 53

**Q:**

Ce puteți face ca programator pentru a preveni deadlock-urile?

**A:**

- Folosiți o ordine consistentă de blocare
  - Evitați blocaje imbricate
  - Utilizați ierarhii de resurse
- 

## Întrebarea 54

**Q:**

Prin ce tranziție de stare va trece un process când citește dintr-un fișier?

**A:**

Când un proces citește dintr-un fișier, trece din starea de execuție (running) în starea blocată (blocked/wait) dacă datele nu sunt imediat disponibile, și revine în ready și apoi running odată ce datele sunt disponibile.

---

## Întrebarea 55

**Q:**

Ce conține superblocul unui disc Linux?

**A:**

filesystem type(ext4, ext3), size information(blocks, free blocks, size), state information, inode information, uuid, mount information

In curs scrie:

Blocul 1 este numit și superbloc. In el sunt trecute o serie de informații prin care se definește

sistemul de fișiere de pe disc. Printre aceste informații amintim:

- numărul n de inoduri (detaliem imediat);
- numărul de zone definite pe disc;
- pointeri spre harta de biți a alocării inodurilor;

- pointeri spre harta de biți a spațiului liber disc;
  - dimensiunile zonelor disc, etc.
- 

## Întrebarea 56

Q:

Se poate crea un link hard spre un fișier aflat pe o altă partiție? Justificați răspunsul.

A:

Nu, deoarece hard link-urile trebuie să fie pe același sistem de fișiere, deoarece referențiază i-node-ul, iar i-node-urile sunt unice doar în cadrul sistemului lor de fișiere.

---

## Întrebarea 57

Q:

Dați o expresie regulată care acceptă orice număr impar de cuvinte separate prin spații, fiecare cuvânt având număr impar de litere.

A:

`^([a-z]{2}*[a-z] [a-z]{2}+ ){0,}([a-z]{2}*[a-z] [a-z]{2}+)*$` -asta a mers pe terminal

Solutia mea (merge pe terminal):

```
grep -E -i "^(([a-z]){2}*[a-z] ){2}*([a-z]){2}*[a-z]$" a.txt
<\(...)*.\> (<\(...)*.\> <\(...)*.\>)*
```

---

## Întrebarea 58

Q:

Dați patru comenzi care afișează numărul de linii goale dintr-un fișier.-8, 8

A:

```
grep -c "^$" filename
grep -Ec "^$" filename
awk "BEGIN {count = 0} /^$/ {count++} END {print count}" filename
sed -n "/^$/p" filename | wc -l
Merge si asta:
grep -E -v -c "." filename
grep -E -c "^$" a.txt
grep -E -V -c "..*" a.txt
grep -E -v -c ".*" a.txt
grep -E -v -c ".{1,}" a.txt
```

---

## Întrebarea 59

Q:

Scrieți o comandă SED care afișează liniile dintr-un fișier ștergând din ele primul, al treilea, al cincilea, al șaptelea, etc spații.

**A:**

```
sed -E "s/([^\ ]*) ([^\ ]*) \1 \2/g' filename (no idea??)
```

```
sed -E "s/([^\ ]*)([^\ ]*)([^\ ]*)\1\3\4/gi" a.txt (pare ca merge pe terminal)
```

```
sed -E "s/([^\ ]+)?([^\ ]+)?\1\3/gi" test.txt
```

---

## Întrebarea 60

**Q:**

Scrieți o comandă AWK care afișează produsul ultimului câmp al liniilor de pe poziții impare care au număr impar de câmpuri.

**A:**

```
awk 'BEGIN {p=1} NR%2==1 && NF%2==1 {p *= $NF} END {print p}' fisier
```

Explicație:

- BEGIN {p=1} = inițializează p = 1 (produs inițial)
- NR%2==1 = doar liniile de pe poziție impară (NR = Number of Record)
- NF%2==1 = doar liniile cu număr impar de câmpuri (NF = Number of Fields)
- {p \*= \$NF} = înmulțește p cu ultimul câmp (\$NF = last field)
- END {print p} = la final afișează produsul

Atenție: folosiți apostrof simplu (') în jurul programului AWK, nu ghilimele duble – altfel shell-ul va încerca să interpreteze \$NF.

---

## Întrebarea 61

**Q:**

Dați patru moduri prin care ieșirea standard a unui proces poate fi redirectată.

**A:**

```
command > file
```

```
command >> file
```

```
command | file
```

```
command > file 2>&1
```

Asta nu mi merge pe terminal:

```
command | file (ok, am revenit si cred ca merge, doar trebuia sa o scriu altfel:)) )
```

Poate asta: (?)

```
command 2> a.txt
```

Nu stiu, mai merge intebat pe cineva care stie:))

Redirect it via > : command > a.txt

Redirect it via >> : command >> a.txt

Redirect it via pipe : `command | echo > a.txt`

Redirect it with tee: `echo "Hello" | tee greetings.txt`

---

## Întrebarea 62

**Q:**

Scrieți trei condiții Shell UNIX care verifică existența unui fișier.

**A:**

```
if test -e fisier; then
    echo "Există"
fi
```

```
if [ -e fisier ]; then
    echo "Există"
fi
```

```
if [[ -e fisier ]]; then
    echo "Există"
fi
```

Alte variante:

- 1) `if [ -e $fisier ]`
  - 2) `if [ $(ls $fisier 2>&1 | grep -E -c "No such file") -eq 0 ]`
  - 3) `if $(ls $fisier 2>&1 | grep -E -q "No such file")`
- 

## Întrebarea 63

**Q:**

Adăugați codul C necesar pentru ca instrucțiunea de mai jos să nu se blocheze așteptând la intrarea standard.

```
execlp("cat", NULL);
```

**A:**

```
int fd = open(STD_IN, "O_WRONLY", 0644)
dup2(fd, STDIN_FILENO);
(cred???)
```

Nu stiu daca e bine. Eu am testat asta pe terminal si merge: `execlp("sh", "sh", "-c", "cat a.txt > b.txt", NULL);`  
`execlp("cat", "cat", "filename", NULL);`  
`(cred???)x2`

```
int fd = open("a.txt", O_RDWR);
dup2(fd, 0);
execlp("cat", "cat", NULL);
```

---



## Întrebarea 64

Q:

Schițați o implementare a funcțiilor `popen` și `pclose`, doar pentru cazul în care outputul comenzii e citit în codul C.

A:

`popen`:

- creează un pipe
- face fork procesului
- copil: închide capătul de citire al pipe-ului, redirectează `stdout` către pipe și execută comanda cu `execlp`
- părinte: închide capătul de scriere al pipe-ului, returnează un pointer `FILE*` pentru citire

`pclose`:

- închide pointer-ul de fișier
  - așteaptă procesul copil
  - returnează statusul de ieșire al copilului
- 

## Întrebarea 65

Q:

Câte FIFO-uri poate deschide pentru citire un process, dacă FIFO-urile sunt și vor fi întotdeauna folosite de alte procese doar pentru citire?

A:

Depinde. Comportamentul special al FIFO este că așteaptă să fie deschis pentru operația complementară. Dacă fiecare dintre celelalte procese deschide exact un FIFO, atunci putem deschide câte vrem. Altfel, dacă nu suntem atenți, poate duce la deadlock.

Să presupunem că avem FIFO-urile 1 și 2. Fie A un proces care deschide FIFO-urile în ordinea 1, 2. Dacă celălalt proces deschide FIFO-urile în ordinea 2, 1, avem deadlock și nu mai putem avansa.

Depinde și de limita descriptorilor de fișier a sistemului, care poate fi verificată cu `"ulimit -n"`.

---

## Întrebarea 66

Q:

Când ați folosi un FIFO în locul unui pipe?

A:

Aș folosi un FIFO în loc de pipe atunci când trebuie să lucrez cu procese care provin din programe diferite și care nu au un strămoș comun. Pentru ca cele două să poată comunica, ar avea nevoie de un fișier pe disc, ceva ce un pipe nu poate oferi, așa că FIFO-ul ar fi soluția.

Diferențele dintre FIFO și pipe:

- FIFO-urile sunt fișiere pe disc, deci au un ID unic la nivel de sistem (calea fișierului) pe care procesele îl pot folosi pentru a le adresa.
  - Crearea pipe-ului îl și deschide, dar FIFO-urile trebuie create și deschise explicit.
  - Pipe-urile sunt distruse când toate capetele sunt închise; FIFO-urile trebuie șterse explicit.
  - FIFO-urile pot trăi dincolo de procesele care le folosesc.
- 

## Întrebarea 67

Q:

Ce este o "secțiune critică"?

A:

O secțiune critică este o parte a programului care accesează resurse partajate și le modifică într-un mod în care modificările nu sunt efectuate în operații atomice. Pentru această secțiune, resursele partajate trebuie protejate pentru a evita accesul concurent. Această secțiune protejată nu poate fi accesată de mai mult de un proces/thread o dată.

O secțiune critică conține operații care nu sunt atomice, deci trebuie protejată folosind un mecanism de sincronizare cum ar fi mutex-uri, read-write locks, semafoare etc.

---

## Întrebarea 68

Q:

Când ați folosi un mutex în locul unui rwlock?

A:

- Când un program are mai multe operații de scriere decât de citire, mutex-ul ar fi mai eficient.
- Simplitate.
- Evitarea deadlock-urilor: mutex-urile sunt mai puțin predispuse la deadlock-uri.

Aș folosi un mutex în loc de rwlock când lucrez cu variabile condiționale (pthread\_cond\_wait poate elibera doar mutex-uri, nu rwlock-uri). De asemenea, read locks sunt utile doar când un thread efectuează operații exclusiv de citire; dacă toate thread-urile fac operații neatomice, un rwlock nu se justifică.

---

## Întrebarea 69

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_wait`?

`pthread_mutex_lock` with calls to `sem_wait`?

**A:**

`sem_wait` decrementează semaforul și poate permite mai multor thread-uri să intre în secțiunea critică, ceea ce ar duce la race conditions și corupere de date.

Trei cazuri:

- 1) Înlocuim doar `sem_wait` cu `mutex_lock`, dar nu și `mutex_unlock` cu `sem_post` → comportament nedefinit cauzat de deblocarea unui mutex care nu a fost blocat.
- 2) Folosim un semafor binar → la fel ca un mutex; la fiecare trecere semaforul este decrementat la 0, iar la `sem_post` este incrementat.
- 3) Folosim un semafor generic cu x thread-uri alocate → thread-urile trec până când semaforul permite, iar în funcție de operațiile executate rezultatele pot fi imprevizibile.

---

## Întrebarea 70

**Q:**

Ce face `pthread_cond_wait` cu mutex-ul primit ca argument?

`pthread_cond_wait` do with the mutex it gets as argument?

**A:**

- Eliberează mutex-ul, permițând altor thread-uri să-l obțină.
- Pune thread-ul "la somn" și așteaptă să fie semnalizată variabila condițională.
- Reachiziționează mutex-ul, asigurând acces exclusiv la resursele partajate.

`pthread_cond_wait` deblochează mutex-ul când este apelată. Când e trimis un semnal (prin signal sau broadcast), `pthread_cond_wait` așteaptă ca mutex-ul să fie liber, îl achiziționează și îl blochează, apoi continuă execuția programului.

---

## Întrebarea 71

**Q:**

Schițați o soluție pentru problema producător-consumator.

**A:**

Răspuns:

N = dimensiunea buffer-ului

p = următorul index pentru inserare obiect

c = următorul index pentru extragere obiect

Consumator: ia obiecte cât timp nu e buffer-ul gol ( $p == c$ ).

Producător: produce obiecte cât timp nu e buffer-ul plin ( $(p+1) \% N == c$ ).

Folosim 2 semafoare:

- unul cu valoare inițială 0 (indică sloturile ocupate)
- unul cu valoare inițială N (indică sloturile libere)
- plus un mutex (sau semafor binar) pentru excluderea mutuală la modificarea buffer-ului.

---

## Întrebarea 72

Q:

Ce puteți face ca programator pentru a preveni deadlock-urile?

A:

- Folosiți o ordine consistentă de blocare.
- Evitați blocaje imbricate.
- Folosiți algoritmi de detectare a deadlock-urilor.
- Folosiți timeout-uri pentru blocare.

---

## Întrebarea 73

Q:

Prin ce tranziție de stare va trece un process când apelează `pthread_cond_wait`? Justificați răspunsul.

`pthread_cond_wait`? Justify your answer.

A:

- Când `pthread_cond_wait` este apelată, thread-ul eliberează mutex-ul asociat și se blochează.
- Rămâne blocat până când un alt thread semnalizează condiția cu `pthread_cond_signal` sau `pthread_cond_broadcast`.
- După ce condiția este semnalizată, thread-ul este trezit, reacheziționează mutex-ul și revine în starea RUNNING.

Un semafor este un întreg a cărui valoare nu poate scădea sub zero. Două operații pot fi efectuate pe semafoare: incrementarea (`sem_post`) și decrementarea (`sem_wait`). Dacă valoarea unui semafor este 0, `sem_wait` va bloca thread-ul până când valoarea devine mai mare decât 0.

Când un proces apelează `pthread_cond_wait`, va trece din RUN în WAIT. Acest comportament este cauzat de faptul că, din moment ce thread-ul va aștepta un semnal de la alt thread, procesorul ar fi ocupat fără motiv, așa că eliberează core-ul pentru ca alt proces să-și completeze taskurile.

---

## Întrebarea 74

Q:

Ce conține un fișier de tip director în sistemul de fișiere Linux?

A:

Intrări de director (nume de fișiere, numere de i-node):

- "." (punct) reprezintă directorul însuși, are i-node-ul directorului
  - ".." (două puncte) reprezintă directorul părinte, i-node-ul părintelui
  - intrări obișnuite: nume de fișier sau subdirector și i-node-ul specific.
- 

## Întrebarea 75

Q:

Explicați diferența dintre un link simbolic și un link hard.

A:

Link simbolic (soft link):

- indică spre calea fișierului țintă
- poate traversa sisteme de fișiere diferite
- devine invalid dacă fișierul țintă este șters
- creat cu "ln -s"

Hard link:

- indică spre i-node-ul fișierului țintă
  - nu poate traversa sisteme de fișiere diferite
  - rămâne valid atâta timp cât există cel puțin un link
  - creat cu "ln"
- 

## Întrebarea 76

Q:

Scrieți o expresie regulată care acceptă orice linie care conține cel puțin trei vocale.

A:

```
[aeiouAEIOU].*[aeiouAEIOU].*[aeiouAEIOU]  
^(.*[aeiouAEIOU]){3}.*$
```

---

## Întrebarea 77

Q:

Care este principiul localității cu privire la încărcarea paginilor?

**A:**

Un proces va avea nevoie probabil în curând de paginile vecine paginii tocmai încărcate. Prezici de ce ar avea nevoie programul și, când încarci o pagină, preîncarci și vecinii ei.

Când încărcăm o pagină, încărcăm și paginile din jurul ei, pentru că probabil vom avea nevoie de ele în curând.

---

## Întrebarea 78

**Q:**

Planificați joburile de mai jos (Nume/Durată/Termen-limită) astfel încât suma întârzierilor să fie minimizată.

A/5/7 B/2/4 C/4/13 D/3/8

**A:**

B, A, D, C

We have for D a delay of 2, and for C a delay of 1. Total delay is 3

---

## Întrebarea 79

**Q:**

Care este riscul adus de funcția `f` când e executată în mai multe thread-uri simultane? Justificați răspunsul.

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    ...
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

**A:**

Există risc de deadlock, deoarece thread-urile nu blochează cele 2 mutex-uri în aceeași ordine.

De exemplu, dacă există 2 thread-uri cu id-urile 0 și 1:

Thread 0: `pthread_mutex_lock(&m[0]);`

Thread 1: `pthread_mutex_lock(&m[1]);`

Să presupunem că amândoi reușesc să achiziționeze mutex-ul. Apoi:

Thread 0 are mutex 0, Thread 1 are mutex 1.

La pasul următor, ar încerca să execute:  
Thread 0: pthread\_mutex\_lock(&m[1]);  
Thread 1: pthread\_mutex\_lock(&m[0]);

Deoarece Thread 1 are mutex 1, Thread 0 ar aștepta. Deoarece Thread 0 are mutex 0, Thread 1 ar aștepta. Astfel, ambele așteaptă unul după altul → deadlock.

---

## Întrebarea 80

Q:

Dați un avantaj și un dezavantaj al politicii de plasare First Fit față de politica Worst Fit.

A:

Avantaj: First Fit este mai rapid decât Worst Fit, deoarece nu trebuie să parcurgă toată lista de blocuri libere.

Dezavantaj: Are mai multe șanse să lase mici găuri în memorie => fragmentare a memoriei.

---

## Întrebarea 81

Q:

Considerând că dimensiunea unui bloc este B și dimensiunea unei adrese este A, câte blocuri de date sunt adresate de indirectarea triplă a unui i-node?

A:

$$(B/A) * (B/A) * (B/A) = B^3/A^3$$

De ce? Pentru că avem 3 niveluri de indirectare, iar fiecare nivel este un bloc de dimensiune B/A. Astfel, avem B/A blocuri în primul nivel, B/A blocuri în al doilea nivel și B/A blocuri în al treilea nivel.

---

## Întrebarea 82

Q:

Scrieți o expresie regulată care acceptă linii care conțin "a", dar nu "b".

A:

$^{\wedge}[^{\wedge}b]^*a[^{\wedge}b]^*\$$

Sau folosind lookahead negativ:

$^{\wedge}(?!.*b).^*a.*\$$

Explicație:

(?!.\*b) => lookahead negativ, înseamnă că expresia nu se va potrivi dacă linia conține "b"

.\*a.\* => se potrivește cu orice linie care conține "a"

.\*\$ => se potrivește cu sfârșitul liniei

Variantă cu grep:

grep "a" file\_name | grep -v "b"

---

## Întrebarea 83

Q:

Care este numărul maxim de procese copil, create de codul de mai jos, care pot coexista simultan?

```
for(i=0; i<7; i++) {  
    if (fork() == 0) {  
        sleep(rand() % 10);  
        exit(0);  
    }  
    if(i % 3 == 0) {  
        wait(0);  
    }  
}
```

A:

5

i = 0 => Se creează C1 => 1 proces copil; așteaptă să se termine => 0 copii

i = 1 => Se creează C2, nu așteaptă => 1 copil

i = 2 => Se creează C3, nu așteaptă => 2 copii

i = 3 => Se creează C4 => 3 copii; așteaptă să se termine => 2 copii

i = 4 => Se creează C5 => 3 copii

i = 5 => Se creează C6 => 4 copii

i = 6 => Se creează C7 => 5 copii; așteaptă => 4 copii

Maxim simultan: 5 copii.

---

## Întrebarea 84

Q:

Câte thread-uri ați folosi pentru procesarea unui milion de fișiere? Justificați răspunsul.

A:

Să presupunem că avem k core-uri (nuclee). Atunci putem folosi k thread-uri pentru procesarea fișierelor. Asta deoarece numărul de thread-uri ar trebui să fie egal cu numărul de core-uri, pentru a folosi toate core-urile în același timp.



Dacă folosim mai multe thread-uri decât core-uri, thread-urile vor trebui să aștepte unele după altele (context switch frecvent), iar asta nu va fi eficient.

Dacă folosim mai puține thread-uri decât core-uri, nu vom folosi toate core-urile disponibile.

---

## Întrebarea 85

Q:

De ce o operație I/O cauzează un proces să treacă din starea running în starea waiting?

A:

Deoarece operațiile I/O pot dura mult, planificatorul (scheduler) pune procesul în starea Waiting și rulează alt proces care e în starea Ready. Acest lucru permite o utilizare mai eficientă a procesorului — în loc să stea inactiv așteptând I/O, CPU-ul lucrează pentru un alt proces. (Scheduler-ul este preemptiv, anticipativ.)

---

## Întrebarea 86

Q:

Cum se face calculul adresei în alocarea cu partiții fixe absolute?

A:

Adresa este aceeași cu adresa fizică. Sistemul de operare nu face nicio translatare.

În alocarea cu partiții fixe absolute, există un număr predefinit de partiții în RAM care pot fi folosite de un program. Acest număr de partiții este fix, iar un program este compilat pentru o anumită partiție.

Aceasta înseamnă că adresa finală executabilă a unei variabile în RAM este cunoscută la compilare, deci adresa din binar este aceeași cu adresa finală în executabil:

Adresa binară = Adresa fizică.

La compilare, programul folosește adresa de început a partiției pentru care este compilat ca să determine adresa finală exactă.

---

## Întrebarea 87

Q:

Procesele A, B și C comunică prin FIFO-urile X, Y și Z conform diagramei de mai jos. Schițați fragmentele de cod care deschid FIFO-urile în cele trei procese.

A --X--> B

B --Y--> C

C --Z--> A

A:

```
//A.cpp
int a2b = open("X", O_WRONLY);
int c2a = open("Z", O_RDONLY);
```

```
//B.cpp
int a2b = open("X", O_RDONLY);
int b2c = open("Y", O_WRONLY);
```

```
//C.cpp
int b2c = open("Y", O_RDONLY);
int c2a = open("Z", O_WRONLY);
```

InA

Open de write pt X

Open de read pt Z

Inb

Open de write pt Y, si ree pt X

InC open de read pt Y, write pt Z si

---

## Întrebarea 88

Q:

15)

Dati un exemplu de valori pentru T, N1, N2 si N3 pentru care programul de mai jos se incheie  
pthread\_barrier\_t bl, b2, b3;

```
void* f1(void® a) {
pthread_barrier_wait(&bl);
return NULL;
```

```
void* f2 (void a) {
pthread_barrier_wait(&b2);
return NULL;
```

»

```
void* f2 (void a) {
pthread_barrier_wait(&b3);
return NULL;
```

?

```
int main() {
int 4;
pthread t t[T][3]5
```

```

pthread_barrier_init(&b1, N1);
pthread_barrier_init(&b2, N2);
pthread_barrier_init(&b3, N3);

for (1-05 i<Ts is) {
pthread_create(St{il[0], NULL, #1,
pthread_create(St{ij[1], NULL, f2,
pthread_create(St[1][2], NULL, f3,
?

for (i=05 icTs t+) f
pthread_join(t{i}[@], NULL);
pthread_join(t{i}[1], NULL);
pthread_join(t{i}[2], NULL);

x
pthread_barrier_destroy(&b1);
pthread_barrier_destroy(&b2);
pthread_barrier_destroy (&b3);

return NULL;

NULL);
NULL);
NULL);

```

**A:**

$T = N1 = N2 = N3 = \text{orice valoare} > 0$

Explicație: codul creează T thread-uri pentru fiecare dintre f1, f2, f3 (deci T thread-uri ajung la fiecare barieră). Bariera bi se eliberează când Ni thread-uri ajung la ea.

Condiția pentru ca programul să se termine: T trebuie să fie multiplu al lui N1, N2 și N3 simultan.

Valori simple:  $T = N1 = N2 = N3 = 3$  (sau orice alt număr egal).

Valori distincte:  $T = 12, N1 = 2, N2 = 3, N3 = 4$ .

## Întrebarea 89

**Q:**

Câte procese va crea codul de mai jos, excluzând procesul părinte?

```

for(i=0; i<8; i++) {
    if(fork() && i % 2)
        break;
}

```

**A:**

9 child processes

i = 0

Creates C1 => 1 child process

i = 1

P creates C2 => 2 child processes

P breaks

C1 creates C3 => 3 child processes

C1 breaks

i = 2

C2 creates C4 => 4 child processes

C3 creates C5 => 5 child processes

i = 3

C2 creates C6 => 6 child processes

C2 breaks

C3 creates C7 => 7 child processes

C3 breaks

C4 creates C8 => 8 child processes

C4 breaks

C5 creates C9 => 9 child processes

C5 breaks

---

## Întrebarea 90

Q:

Ce va afișa în consolă fragmentul de cod de mai jos, considerând că thread-urile se creează fără probleme? Justificați răspunsul.

```
/* Considerați că header-ele necesare sunt incluse aici */
void* f(void* a) {
    printf("%d\n", *(int*)a);
    return NULL;
}

int main() {
    /* Considerați că variabilele necesare sunt declarate aici */
    for(i=0; i<10; i++) {
        pthread_create(&t[i], NULL, f, &i);
    }
    /* Considerați că aici se fac join-urile necesare */
    return 0;
}
```

}

**A:**

Nu este cert ce vor afișa thread-urile. Output-ul va fi alcătuit din 10 numere, fiecare între 0 și 10, dar ordinea și valorile exacte nu sunt certe.

Aceasta deoarece thread-urile primesc adresa lui  $i$  ca parametru, dar  $i$  este incrementat în timpul buclei for. Astfel, nu putem determina ce se întâmplă: poate  $i$  este incrementat de mai multe ori înainte să fie afișat de un thread, sau poate mai multe thread-uri afișează același  $i$ . Poate un thread care a citit valoarea lui  $i$  într-un moment așteaptă, iar alt thread ajunge să afișeze un  $i$  mai mare înainte ca thread-ul curent să poată afișa valoarea.

---

## Întrebarea 91

**Q:**

Planificați joburile de mai jos (date ca Nume/Durată/Termen-limită) astfel încât suma întârzierilor să fie minimizată.

A/3/8, B/10/15, C/3/5

**A:**

Ordinea: CAB (întârziere totală = 1 secundă)

C: pornește la 0, termină la 3, întârziere = 0 (termen 5)

A: pornește la 3, termină la 6, întârziere = 0 (termen 8)

B: pornește la 6, termină la 16, întârziere = 1 (termen 15)

Întârziere totală = 1.

---

## Întrebarea 92

**Q:**

Date două cache-uri set-asociative, una cu 2 seturi de 4 pagini și alta cu 4 seturi de 2 pagini, care ar performa mai bine pentru secvența de cereri de pagini de mai jos? Justificați răspunsul.

17, 2, 37, 6, 9

**A:**

Pentru cache-uri set-asociative, ca să plasăm o pagină în cache, calculăm mai întâi numărul paginii modulo numărul de seturi, apoi căutăm un loc liber în acel set.

Prima variantă: 2 seturi de 4 pagini.

$17 \bmod 2 = 1 \rightarrow$  în setul 1

$2 \bmod 2 = 0 \rightarrow$  în setul 0

$37 \bmod 2 = 1 \rightarrow$  în setul 1

$6 \bmod 2 = 0 \rightarrow$  în setul 0

$9 \bmod 2 = 1 \rightarrow$  în setul 1

A doua variantă: 4 seturi de 2 pagini.

$17 \bmod 4 = 1$

$2 \bmod 4 = 2$

$37 \bmod 4 = 1$

$6 \bmod 4 = 2$

$9 \bmod 4 = 1 \rightarrow$  ambele pagini din acest grup sunt pline, cache-ul trebuie să elimine o pagină.

Prima variantă (2 seturi de 4 pagini) ar performa mai bine pentru această secvență, deoarece nu trebuie să dea afară o pagină din cache pentru a încărca alta din RAM.

---

### Întrebarea 93

**Q:**

Considerând că un bloc poate conține N adrese către alte blocuri, câte blocuri de date sunt adresate de indirectările dublă și triplă ale unui i-node împreună?

$N*N + N*N*N$

**A:**

De ce? Pentru că indirectarea dublă indică spre N blocuri, fiecare indicând spre alte N blocuri (deci  $N*N$ ), iar indirectarea triplă indică spre N blocuri, fiecare indicând spre N blocuri, fiecare indicând spre alte N blocuri (deci  $N*N*N$ ).

---

### Întrebarea 94

**Q:**

Dați un avantaj și un dezavantaj al alocării cu partiții fixe față de alocarea cu partiții relocatabile.

**A:**

Avantaj: Calculul adreselor este mai ușor, deoarece știm unde se află fiecare proces în memorie.

Dezavantaj: Nu putem folosi memoria eficient, deoarece s-ar putea să rămână spațiu nefolosit dacă un proces nu încapă într-o partiție, ducând la fragmentare a memoriei.

---

### Întrebarea 95

**Q:**

Dați un avantaj și un dezavantaj al încărcării tuturor paginilor unui proces în memorie odată, comparativ cu încărcarea la cerere.

**A:**

Avantaj: Când încercăm să accesăm o pagină, ea este deja în memorie, deci nu trebuie să așteptăm să fie încărcată.

Dezavantaj: S-ar putea să încărcăm pagini de care nu avem nevoie, irosind memorie. De asemenea, durează mai mult să încarci toate paginile odată.

---

## Întrebarea 96

**Q:**

Dați un avantaj și un dezavantaj al cache-ului cu acces direct comparativ cu cache-ul asociativ.

**A:**

Avantaj: Este mai rapid să cauți o pagină în cache, deoarece știm exact unde să o căutăm.

Dezavantaj: Nu putem avea mai multe pagini cu același index în cache, deci s-ar putea să fim forțați să eliminăm o pagină chiar dacă mai e necesară (cache thrashing).

---

## Întrebarea 97

**Q:**

Când își schimbă un proces starea din WAIT în RUN?

**A:**

Un proces își schimbă starea din WAIT în RUN (sau, mai exact, mai întâi în READY) când evenimentul sau condiția pe care o aștepta se întâmplă sau devine disponibilă. Procesul trece dintr-o stare blocată/de așteptare într-o stare gata-de-rulare când devine eligibil pentru a fi planificat la execuție de către sistemul de operare.

---

## Întrebarea 98

**Q:**

De ce se poate crea un hard-link către fișiere de pe aceeași partiție, dar nu către fișiere de pe partiții diferite?

**A:**

Un hard-link indică spre același i-node ca și fișierul. Deoarece i-node-ul este o structură de date folosită pentru a reprezenta un obiect al sistemului de fișiere, este intern sistemului de fișiere și nu poți indica spre un i-node al altui sistem de fișiere.

Fiecare sistem de fișiere are propria organizare și structuri de date pentru gestionarea fișierelor și directoarelor (superblock, tabel de i-noduri, blocuri de date). Diferite partiții pot avea tipuri și configurări diferite de sisteme de fișiere.

Soft link-urile (linkuri simbolice) pot traversa sistemele de fișiere, deoarece doar indică spre o altă intrare de director (interfața sistemului de fișiere, nu un obiect intern).

---

## Întrebarea 99

**Q:**

Adăugați codul sursă necesar la fragmentul de mai jos pentru ca printf să afișeze în consolă.

```
int p[2];
pipe(p);
dup2(p[1], 1);
printf("asdf\n");
```

**A:**

```
#include <stdio.h>
#include <unistd.h>
```

```
int main() {
    int p[2];
    pipe(p);

    if (fork() == 0) {
        // Proces copil
        close(p[1]); // Închide capătul de scriere
        dup2(p[0], 0); // Redirecționează capătul de citire la stdin

        char buffer[256];
        fgets(buffer, sizeof(buffer), stdin);
        printf("%s", buffer);

        close(p[0]);
    } else {
        // Proces părinte
        close(p[0]); // Închide capătul de citire
        dup2(p[1], 1); // Redirecționează stdout la capătul de scriere
        printf("asdf\n");
        close(p[1]);
    }
    return 0;
}
```

---

## Întrebarea 100

**Q:**

Care sunt elementele unei adrese virtuale în alocarea paginată-segmentată și ce tabele sunt implicate în calculul adresei fizice?

**A:**



Elementele unei adrese virtuale sunt (s, p, d), unde:

- s = numărul segmentului
- p = numărul paginii virtuale în segment
- d = deplasamentul (offset) în pagină

Tabelele implicate sunt:

- tabela de segmente a procesului (pentru fiecare segment indică o tabelă de pagini)
- tabela de pagini (pentru fiecare segment)

Deci adresa fizică se calculează: indexul s din tabela de segmente → ne dă tabela de pagini a segmentului; indexul p din această tabelă → ne dă numărul cadrului fizic; combinat cu d → adresa fizică.

---

## Întrebarea 101

Q:

Dați 4 expresii regulate care specifică de câte ori ar trebui să apară expresia anterioară.

a\* -> de 0 sau mai multe ori

a+ -> de 1 sau mai multe ori

a? -> de 0 sau 1 ori

A:

a{x,y} -> între x și y ori, inclusiv. Dacă x este omis, este 0. Dacă y este omis, este infinit.

---

## Întrebarea 102

Q:

Un proces deschide un FIFO pentru scriere și este singurul proces din sistem care va folosi sau va fi folosit acel FIFO. Când își va termina procesul execuția?

A:

Procesul nu își va termina niciodată execuția, deoarece va fi blocat pentru totdeauna. Procesul va fi blocat până când alt proces deschide FIFO-ul pentru citire. Citirea și scrierea într-un FIFO sunt operații complementare; dacă nu există proces care să citească din FIFO, procesul care scrie va fi blocat.

---

## Întrebarea 103

Q:

4 reguli pentru expresii regulate care arată de câte ori trebuie să se repete expresia anterioară:

A:

a\* -> de 0 sau mai multe ori

a+ -> de 1 sau mai multe ori

a? -> de 0 sau 1 ori  
a{x,y} -> între x și y ori, inclusiv

---

## Întrebarea 104

Q:

3. Un proces deschide fifo pt citire si e singurul proces ce va folosi vreodata acel fifo, cand isi va incheia executia?

A:

Niciodata, pentru ca acesta asteapta un alt proces pe cealalta parte a fifo-ului sa citeasca ce scrie, dar acesta

nu va exista.

---

## Întrebarea 105

Q:

1. scrieti un grep care ia grupurile de cate 2 cuvinte, separate de un singur spatiu, care sunt formate doar din litere mici si fiecare cuvant contine cel putin 2 vocale

A:

```
grep -Eo '[bcdfghijklmnpqrstvwxyz]*[aeiou][bcdfghijklmnpqrstvwxyz]*[aeiou]
[bcdfghijklmnpqrstvwxyz]* [bcdfghijklmnpqrstvwxyz]*[aeiou][bcdfghijklmnpqrstvwxyz]*[aeiou]
[bcdfghijklmnpqrstvwxyz]*' grep.txt
"</[a-z]*[aeiou][a-z]*[aeiou][a-z]*/> </[a-z]*[aeiou][a-z]*[aeiou][a-z]*/>"
```

---

## Întrebarea 106

Q:

2. scrieti 2 grep uri care iau liniile care nu au numarul de caractere multiplu al lui 3

A:

```
grep -E -v "^(...)*$" grep.txt
grep -E -v "^(.{3})*$" grep.txt
grep -E -v "^(...)*$"
grep -E -v -i "^(...)*$"da
```

---

## Întrebarea 107

Q:

3. scrieti un sed care inlocuieste prima aparitie a caracterului A cu caracterul B

A:

```
sed "s/A/B/" (de pe fiecare linie?)  
sed -E "s/([^A]*) (A)(.*$)/\1B\3/gi" a.txt  
sed -E "s/A/B/i"
```

---

## Întrebarea 108

Q:

4. scrieti un awk care afiseaza liniile care au primul cuvant identic cu ultimul cuvant si al caror penultim cuvant are numar par de caractere

A:

```
awk 'NF > 1 && $1 == $NF && length($(NF - 1)) % 2 == 0 {print $0}'
```

---

## Întrebarea 109

Q:

5. scrieti 3 moduri de a crea un fisier gol

A:

```
nano file
```

```
> file
```

```
touch file
```

```
echo "" > a.txt
```

```
touch a.txt
```

```
echo "" > a.txt
```

```
echo "" 2> a.txt
```

---

## Întrebarea 110

Q:

6. scrieti 5 moduri de a verifica daca un string este gol(cu test)

A:

```
if test -z "$string";  
if ! test -n "$string"  
if test "$string" == "";  
if test ${#string} -eq 0;  
if test ${#A} -lt 1;
```

```
1) if test -z $A
```

```
2) if test $A=""
```

```
3) if test ${#A} -eq 0
```

```
4) if test ${#A} -lt 1
5) if test ! ${#A} -gt 0
6) if test $(echo $A | grep -E -c "^$") -eq 1
7) if $(echo $A | grep -E -q "^$")
8) if test $(echo $A | awk '{print length}') -eq 0
```

---

## Întrebarea 111

Q:

7. afisati ierarhia proceselor a urmatorului cod:

```
for(int i = 0; i < 3; i++)
    if (fork() != 0)
        wait();
```

A:

Ierarhia este un LANȚ:  $P \rightarrow C1 \rightarrow C2 \rightarrow C3$

Explicație:

- $i=0$ : P face fork  $\rightarrow$  C1. P (fork != 0) face wait. C1 (fork == 0) continuă bucla.
- $i=1$ , în C1: fork  $\rightarrow$  C2. C1 face wait. C2 continuă.
- $i=2$ , în C2: fork  $\rightarrow$  C3. C2 face wait. C3 continuă.
- C3 iese din buclă și se termină.
- C2 e deblocat din wait, se termină.
- C1 e deblocat, se termină.
- P e deblocat, se termină.

Fiecare proces are exact un copil, formând un lanț.

---

## Întrebarea 112

Q:

Ce afișează codul:

```
execlp("expr", "expr", "a", "+", "1", NULL);
printf("xyz\n");
```

A:

Eroare (sau output gol).

expr nu poate calcula "a + 1" pentru că "a" nu e un număr  $\rightarrow$  expr afișează un mesaj de eroare pe stderr.

În ambele cazuri (execlp reușește sau nu):

- Dacă execlp REUȘEȘTE: procesul este înlocuit cu expr. expr rulează, dă eroare (operand invalid) și se termină. printf NU se execută niciodată.

- Dacă `execlp` EȘUEAZĂ (puțin probabil aici): `printf("xyz\n")` s-ar executa și ar afișa "xyz".

Cel mai probabil scenariu: `execlp` reușește, `expr` e rulat, dă eroare → "xyz" nu se afișează niciodată.

---

## Întrebarea 113

Q:

9. schitați o implementare a funcțiilor `popen` și `pclose`

A:

`popen`:

- creează un pipe
- face fork procesului
- copil: închide capătul de citire al pipe-ului, redirectează `stdout` către pipe și execută comanda cu `execlp`
- părinte: închide capătul de scriere al pipe-ului, returnează un pointer `FILE*` pentru citire

`pclose`:

- închide pointer-ul de fișier
- așteaptă procesul copil
- returnează statusul de ieșire al copilului

Implementare:

```
FILE* mypopen(char *cmd, char *type) {
    int p[2];
    pipe(p);
    if ((child_pid = fork()) == 0) {
        if (type[0] == 'r') {
            close(p[0]);
            dup2(p[1], STDOUT_FILENO);
        } else {
            close(p[1]);
            dup2(p[0], STDIN_FILENO);
        }
        execlp("sh", "sh", "-c", cmd, NULL);
        exit(1);
    }
    if (type[0] == 'r') {
        close(p[1]);
        return fdopen(p[0], type);
    } else {
        close(p[0]);
        return fdopen(p[1], type);
    }
}

int mypclose(FILE *fp) {
    int status;
    fclose(fp);
    waitpid(child_pid, &status, 0);
}
```

```
        return status;  
    }
```

---

## Întrebarea 114

Q:

10. cate FIFO pot fi deschise de catre un fisier daca fiecare dintre acele FIFO-uri va avea capatul celalalt deschis de catre un alt proces?

A:

Depinde de limita descriptorilor de fișier ai sistemului, care poate fi verificată folosind "ulimit -n". Este important să deschidem corect și în aceeași ordine FIFO-urile, deoarece altfel poate duce la deadlock-uri. Dacă deschiderea este corectă, limita este cea a descriptorilor de fișier ai sistemului.

---

## Întrebarea 115

Q:

11. cand am dori sa folosim execl si cand am dori sa folosim execv?

A:

Folosim execl: când avem un număr fix și cunoscut de argumente.  
Folosim execv: când numărul de argumente este dinamic sau determinat la runtime.

---

## Întrebarea 116

Q:

12. definiti notiunea de sectiune critica

A:

O secțiune critică este o parte a programului unde avem resurse partajate, cum ar fi variabile de memorie, care pot fi accesate și modificate. Este important să gestionăm programul corect pentru a evita race conditions și deadlock-uri, folosind de exemplu excluderea mutuală (mutex).

---

## Întrebarea 117

Q:

14. care sunt consecintele inlocuirii lui pthread\_mutex\_lock cu sem\_post in cod?

A:

Înlocuirea pthread\_mutex\_lock cu sem\_post ar duce la haos, deoarece sem\_post incrementează semaforul, permițând oricărui thread să intre în secțiunea

critică, eșuând astfel în prevenirea accesului concurent la resursele partajate. Acest lucru ar putea duce la race conditions și corupere de date.

---

## Întrebarea 118

Q:

15. definiți un semafor binar și explicați cum funcționează

A:

Un semafor binar este un mecanism de sincronizare care poate avea doar două valori: 0 și 1.

Operații:

- wait (P / sem\_wait): decrementează semaforul (de la 1 la 0). Dacă semaforul e deja 0, thread-ul se blochează.
- signal (V / sem\_post): incrementează semaforul (de la 0 la 1) și trezește un thread care aștepta.

Funcționează ca un mutex: doar un singur thread poate "intra" la un moment dat.

---

## Întrebarea 119

Q:

17. scrieți un mod de a preveni deadlock

A:

- Folosiți o ordine consistentă de blocare
  - Evitați blocaje imbricate
  - Folosiți algoritmi de detectare a deadlock-urilor
  - Folosiți timeout-uri pentru blocare
- 

## Întrebarea 120

Q:

18. prin ce stare (gen ready, wait, swap, etc) trece un proces când apelăm pthread\_join?

A:

Stare inițială: thread-ul este în starea de execuție (RUNNING) și apelează pthread\_join.

Așteptare terminare: thread-ul care apelează pthread\_join trece în starea blocată (BLOCKED/WAIT), așteptând ca thread-ul țintă să se termine.

Thread-ul țintă se termină: când thread-ul țintă își completează execuția, trece în starea TERMINATED.

Deblocarea: thread-ul care aștepta este deblocat și revine în starea READY (apoi RUNNING).

---

## Întrebarea 121

Q:

19. dacă avem B drept block size și A drept address size, câte adrese o să aibă un double indirect dintr-un i-node?

A:

$(B/A)^2$  adrese (sau blocuri de date).

Un bloc are B octeți, o adresă are A octeți, deci un bloc conține  $B/A$  adrese.

Indirectarea dublă funcționează astfel:

- Un pointer din i-node spre un bloc cu  $B/A$  adrese
- Fiecare dintre acele  $B/A$  adrese indică spre alt bloc cu  $B/A$  adrese
- Fiecare dintre acele adrese indică spre un bloc de date

Total:  $(B/A) \times (B/A) = (B/A)^2$

---

## Întrebarea 122

Q:

20. ce se întâmplă cu conținutul directorului în care montăm o partiție?

A:

Conținutul existent al directorului în care montezi partiția devine temporar inaccesibil. După montare, orice acces la director accesează de fapt rădăcina partiției montate.

---

## Întrebarea 123

Q:

Dați o expresie regulată care se potrivește cu orice secvență de lungime pară de cuvinte cu litere mici separate prin spații, dacă pentru fiecare cuvânt lungimea sa și poziția sa în secvență sunt fie ambele impare, fie ambele pare. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea trebuie să aibă lungime pară.

A:

Pentru cuvinte cu litere mici separate prin spații, unde cuvântul de pe poziția impară are lungime impară și cel de pe poziția pară are lungime pară:



$^{([a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^* )+[a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^* \$}$

Explicație:

- $[a-z]([a-z][a-z])^*$  = cuvânt de lungime impară (1, 3, 5...)
- $[a-z][a-z]([a-z][a-z])^*$  = cuvânt de lungime pară (2, 4, 6...)
- Le combinăm în perechi (impar, par) repetate, pentru a obține lungime pară a secvenței.

---

## Întrebarea 124

Q:

Câte FIFO-uri poate deschide un proces pentru citire, dacă FIFO-urile sunt și vor fi folosite de alte procese doar pentru scriere?

A:

Numărul maxim este limitat doar de numărul de file descriptors permis sistemului (verificabil cu "ulimit -n").

Condiția importantă: pentru fiecare FIFO deschis pentru CITIRE, trebuie să existe cel puțin un proces care să-l deschidă pentru SCRIERE — altfel apelul open() se blochează.

Dacă procesele de scriere folosesc fiecare FIFO-urile în aceeași ordine ca procesul nostru, putem deschide câte vrem (până la limita ulimit -n).

---

## Întrebarea 125

Q:

Explicați de ce descriptorul de fișier returnat de popen trebuie închis cu pclose în loc de fclose.

A:

popen creează un proces copil prin fork + exec și returnează un FILE\* către pipe-ul de comunicare.

Dacă închidem cu fclose:

- Doar stream-ul de date se închide
- Procesul copil rămâne zombie (nu îl așteaptă nimeni)
- Resursele copilului nu se eliberează corect

pclose:

- Închide stream-ul
- Apelează wait() pentru procesul copil (evită zombi)
- Returnează exit status-ul comenzii

Deci pclose face curățenia completă, iar fclose lasă în urmă un zombie.

---

## Întrebarea 126

Q:

Ce va afișa fragmentul de mai jos? Justificați răspunsul.

```
execl("expr", "expr", "1", "+", "1", NULL);
execlp("echo", "echo", "3", NULL);
printf("4\n");
```

A:

Output: 3

Justificare:

- execl("expr", ...) – eșuează pentru că "expr" nu este cale absolută și execl NU caută în PATH. Programul continuă.
  - execlp("echo", "echo", "3", NULL) – reușește (execlp caută în PATH), înlocuiește procesul curent cu echo, care afișează "3" și se termină.
  - printf("4\n") – NU se execută, pentru că execlp deja a înlocuit procesul.
- 

## Întrebarea 127

Q:

Dați trei moduri de a afla dimensiunea unui fișier din linia de comandă Linux.

A:

1. ls -l fișier (a 5-a coloană din output este dimensiunea în octeți)
2. wc -c fișier (numără octeții)
3. du -h fișier (utilizarea pe disc, dimensiune lizibilă)

Alternative:

- stat -c%s fișier (dimensiunea exactă în octeți)
  - ls -s fișier (dimensiune în blocuri)
  - find . -name fișier -printf "%s\n"
- 

## Întrebarea 128

Q:

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență ne-vidă de litere mici.

A:

```
sed 's/[a-z]\+//' fisier  
sed -E 's/[a-z]\+//' fisier
```

Explicație:

- [a-z]+ = una sau mai multe litere mici consecutive
- s/.../... = substituție (implicit doar prima apariție per linie)
- Cu -E (extended regex) nu trebuie escape pe +; fără -E folosim \+

---

## Întrebarea 129

Q:

Scrieți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărei linii constau din secvențe de cifre separate prin spații.

A:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Sau mai simplu, dacă fiecare linie are sumă pe coloana:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Explicație:

- NF = numărul de câmpuri pe linia curentă
- Bucla parcurge toate câmpurile și adună
- END {} se execută o singură dată la final

---

## Întrebarea 130

Q:

Dați trei comenzi GREP care afișează liniile unui fișier care constau exclusiv dintr-o secvență ne-vidă de litere și cifre alternante (ex: a0g sau 1r5m).

A:

```
grep -E '^([a-z][0-9])+|^([0-9][a-z])+$' fisier  
grep -E '^a-z0-9+$' fisier | grep -vE '[a-z]{2}|[0-9]{2}'  
egrep '^([a-z][0-9])+|^([0-9][a-z])+$' fisier
```

Explicație:

- ^...\$ = linia întreagă
- ([a-z][0-9])+ = perechi literă-cifră repetate
- | = sau
- ([0-9][a-z])+ = perechi cifră-literă repetate

---

## Întrebarea 131

Q:

Ce tranziție de stare va suferi un proces când apelează sem\_wait și în ce condiții? Justificați răspunsul.

**A:**

Procesul trece din starea RUNNING în starea WAITING (blocat) atunci când:

- Apelează `sem_wait` pe un semafor cu valoarea 0 (sau care ar deveni negativ după decrementare).

Dacă semaforul are valoarea  $> 0$ , `sem_wait` doar decrementează valoarea și procesul continuă să ruleze (rămâne în RUNNING).

Motiv: dacă procesul ar aștepta activ (busy-wait), ar irosi un core fără să facă nimic. SO îl pune în WAIT și rulează alt proces.

---

## Întrebarea 132

**Q:**

Ce s-ar putea întâmpla dacă funcția `f` ar fi executată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id + 1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

**A:**

Riscul este DEADLOCK.

Thread-urile cu id par ( $id \% 2 = 0$ ) blochează mai întâi `m[0]`, apoi `m[1]`.

Thread-urile cu id impar ( $id \% 2 = 1$ ) blochează mai întâi `m[1]`, apoi `m[0]`.

Dacă două thread-uri (unul par, unul impar) rulează simultan:

- Thread A (id par) prinde `m[0]`
- Thread B (id impar) prinde `m[1]`
- Thread A încearcă să prindă `m[1]` → blocat
- Thread B încearcă să prindă `m[0]` → blocat
- Niciunul nu poate avansa → DEADLOCK

Cauza fundamentală: nu se respectă o ordine fixă de achiziție a mutex-urilor.

---

## Întrebarea 133

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_post`?

A:

Apariție de race conditions și corupere de date.

`sem_post` incrementează valoarea semaforului, dar NU asigură excluderea mutuală. Mai multe thread-uri pot intra în "secțiunea critică" simultan, ceea ce duce la:

- Modificări concurente ale resurselor partajate
- Rezultate impredictibile

În plus, dacă `pthread_mutex_unlock` rămâne în cod, va apela `unlock` pe un mutex care nu a fost niciodată blocat → comportament nedefinit.

---

## Întrebarea 134

Q:

Dați trei apeluri de funcție care asigură excluderea mutuală.

A:

Trei apeluri care asigură excluderea mutuală:

1. `pthread_mutex_lock(&m)`
    - Cel mai folosit. Blochează un mutex, un singur thread poate intra.
  2. `sem_wait(&s)` cu un semafor binar (inițializat cu 1)
    - Echivalent funcțional cu `mutex_lock`.
  3. `pthread_rwlock_wrlock(&rw)`
    - Blocaj de scriere exclusiv pe un `rwlock` – niciun alt thread (nici cititor, nici scriitor) nu mai poate intra.
- 

## Întrebarea 135

Q:

Ce conține un fișier de tip director în sistemul de fișiere Linux? - 2, 8

A:

Un fișier de tip director conține o listă de intrări (directory entries), fiecare având:

- Numele fișierului (sau subdirectorului)
- Numărul i-node-ului asociat

Intrările speciale:

- "." (punct) — directorul însuși (i-node-ul lui propriu)
- ".." (două puncte) — directorul părinte
- Intrările obișnuite — numele fișierelor/subdirectoarelor din director și i-node-urile lor.

Observație: numele fișierului NU e stocat în i-node, ci doar în director. I-node-ul conține metadata (drepturi, dimensiune, pointeri spre date).

---

## Întrebarea 136

**Q:**

Se da fișierul a.log de mai jos în care un server concurent a scris evenimentele aparute pe parcursul rezolvării diferitelor cereri. Fiecare eveniment apare pe o singură linie și are formatul: data ora tip cerere detalii. Scrieți un script Shell UNIX care separă liniile din a.log în fișiere separate (cate unul pentru fiecare cerere) și afișează un top al cererilor în funcție de numărul de erori aparute în procesarea ei.

**A:**

```
#!/bin/bash

# Separă liniile din a.log în fișiere pe tip de cerere

while IFS= read -r linie; do
    # Format: data ora tip cerere detalii
    tip=$(echo "$linie" | awk '{print $3}')
    echo "$linie" >> "log_${tip}.txt"
done < a.log

# Variantă mai eficientă cu awk:
awk '{print >> "log_" $3 ".txt"}' a.log
```

---

## Întrebarea 137

**Q:**

Dați o expresie regulată care acceptă orice secvență de lungime pară de cuvinte (formate din litere) separate prin spații, cu condiția ca pentru fiecare cuvânt să lungimea și poziția în secvență să fie ambele pare sau impare. Numărarea cuvintelor începe de la 1. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea să aibă lungime pară.

**A:**

La fel ca la întrebarea 123, varianta echivalentă:

$^{([a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^* )+[a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^* \$}$

Ideea cheie:

- Cuvânt pe poziție impară → lungime impară →  $[a-z]([a-z][a-z])^*$
- Cuvânt pe poziție pară → lungime pară →  $[a-z][a-z]([a-z][a-z])^*$
- Toate perechile (impar, par) compun secvența de lungime pară.

---

## Întrebarea 138

Q:

Dați trei comenzi GREP care afișează dintr-un fișier liniile formate exclusiv dintr-o secvență nevidă de litere și cifre alternativ (ex: a0g sau 1r5m).

A:

```
grep -E '^[a-z][0-9]+$|^[0-9][a-z]+$' fisier
egrep '^[a-z][0-9]+$|^[0-9][a-z]+$' fisier
grep '^[a-z][0-9]\)\+$|^[0-9][a-z]\)\+$' fisier
```

Doar litere și cifre alternativ, secvență nevidă, întreaga linie.

---

## Întrebarea 139

Q:

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență nevidă de litere mici.

A:

```
sed 's/[a-z]\+//' fisier
sed -E 's/[a-z]\+//' fisier
```

Șterge prima secvență nevidă de litere mici de pe fiecare linie. (Fără /g, doar prima apariție.)

---

## Întrebarea 140

Q:

Scrieți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărei linii sunt formate din secvențe de cifre separate prin spații.

A:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Parcurge toate câmpurile de pe fiecare linie, adună totul, afișează suma la final.

---

## Întrebarea 141

**Q:**

Dați trei moduri de a afla dimensiunea unui fișier în linia de comandă Linux.

**A:**

Trei moduri de a afla dimensiunea unui fișier:

1. `ls -l fisier` (a 5-a coloană este dimensiunea în octeți)
2. `wc -c fisier` (numără octeții)
3. `du -h fisier` (utilizarea pe disc, dimensiune lizibilă)

Alternative: `stat -c%s fisier`, `ls -lh fisier` (lizibil), `find . -name fisier -printf "%s\n"`

---

## Întrebarea 142

**Q:**

Scrieți o condiție Shell UNIX care verifică dacă un fișier există și utilizatorul curent are vreo permisiune asupra lui.

**A:**

```
if [ -e fisier ] && { [ -r fisier ] || [ -w fisier ] || [ -x fisier ] ; } ;  
then  
    echo "Există și am permisiuni"  
fi
```

Sau mai compact:

```
[ -e fisier ] && [ -r fisier -o -w fisier -o -x fisier ]
```

Explicație:

- `-e`: există
  - `-r`: are permisiune de citire
  - `-w`: are permisiune de scriere
  - `-x`: are permisiune de executare
- 

## Întrebarea 143

**Q:**

Ce va tipări fragmentul de cod de mai jos? Justificați răspunsul.

```
execl("expr", "expr", "1", "
```

```
» NULL);
```



```
NULL) ;

execlp("echo"
printf("4\n");

"echo",
```

**A:**

Răspuns: 3

execl nu va executa nimic, deoarece are nevoie de calea completă, iar "expr" nu o are, deci comanda nu poate fi executată. Așa că trecem la execlp, care va afișa "3". execlp înlocuiește procesul curent, deci nu va afișa "4".

Pentru justificare: execl necesită calea completă către executabil (de exemplu execl("/bin/expr", "expr", "1", "+", "1", NULL)). Aici avem doar numele comenzii, deci execl eșuează. Programul continuă la execlp, afișează "3", iar din moment ce execlp înlocuiește imaginea procesului și se termină la finalizare, niciun cod ulterior nu mai este executat.

---

## Întrebarea 144

**Q:**

De ce descriptorul de fișier returnat de popen trebuie închis cu pclose și nu cu fclose?

**A:**

Vezi răspunsul de la întrebarea 125 (popen vs fclose).

Pe scurt: popen creează un proces copil cu fork+exec; pclose închide stream-ul și apelează wait() pentru copil. fclose ar lăsa procesul ca zombie.

---

## Întrebarea 145

**Q:**

Câte FIFO-uri poate deschide pentru citire un process, dacă FIFO-urile sunt și vor fi întotdeauna folosite de alte procese doar pentru scriere?

**A:**

Vezi răspunsul de la întrebarea 124. Pe scurt: câte FIFO-uri permite limita sistemului (ulimit -n), respectând ordinea de deschidere pentru a evita deadlock-uri.

---

## Întrebarea 146

**Q:**

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_post`?

`pthread_mutex_lock` with calls to `sem_post`?

**A:**

Vezi răspunsul de la întrebarea 132.

Înlocuirea `pthread_mutex_lock` cu `sem_post` duce la haos: `sem_post` incrementează semaforul, nu blochează nimic. Mai multe thread-uri pot intra simultan în secțiunea critică → race conditions și corupere de date.

---

## Întrebarea 147

**Q:**

Ce se poate întâmpla dacă funcția `f` este rulată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) %
```

```
pthread_mutex_lock(first) ;
pthread_mutex_lock(second);
```

```
pthread_mutex_unlock(second) ;
```

```
pthread_mutex_unlock(first) ;
```

**A:**

Vezi răspunsul de la întrebarea 131.

Riscul este DEADLOCK pentru că thread-urile cu id par și id impar blochează mutex-urile în ordini opuse. Doi care rulează simultan se pot bloca unul pe altul.

---

## Întrebarea 148

**Q:**

Prin ce tranziție de stare va trece un process când apelează `sem_wait` și în ce condiții? Justificați răspunsul.

**A:**

Vezi răspunsul de la întrebarea 130.

Procesul trece din RUN în WAIT dacă apelează `sem_wait` pe un semafor cu valoarea 0. Dacă semaforul are valoare > 0, procesul rămâne în RUN (doar decrementează valoarea).

---

## Întrebarea 149

Q:

2. Scrieți o comandă UNIX Shell care elimină toate caracterele non-literă din fișierul a.txt (de ex: aB59-\*! -> aB59).

A:

SOLUTION

```
sed -E "s/[^a-z0-9]//gi" a.txt
```

Pha ni lita -: —hlaealaaaetial mee

```
sed -E"s/[*a-z]//gi" a.txt
```

---

## Întrebarea 150

Q:

3. Scrieți un program AWK care, aplicat pe un fișier care conține cuvinte separate prin spații, calculează numărul mediu de cuvinte pe linie.

A:

SOLUTION

```
BEGIN{
    count=0
    wc=0
}
{
    count++
    wc+=NF
}
END{
    print wc/count
}
```

```
avg=words/Lines
int avg
```

---

## Întrebarea 151

Q:

4. Display all the unique file names (without the path) in a given directory and all its hierarchy of subdirectories.

A:

SOLUTION

```
find -type f | awk -F/ '{print $NF}' | sort | uniq  
ls -all | awk '{print $9}' | sort | uniq ??
```

---

## Întrebarea 152

Q:

5. Scrieți un script UNIX Shell care calculează numărul mediu de linii din toate fișierele cu extensia .txt din directorul curent.

A:

```
al $f  
al $f  
(nrfis-3))  
nrfis))  
tru directory $f
```

SOLUTION

```
#!/bin/bash
```

```
nrFiles=0  
nrLines=0  
totalNrLines=0
```

```
for f in `ls | grep -E ".*\.txt$"`; do  
    nrFiles=`expr $nrFiles + 1`  
    nrLines=`cat $f | wc -l`  
    totalNrLines=`expr $totalNrLines + $nrLines`  
done
```

```
result=`expr $totalNrLines / $nrFiles`  
echo "Average is: $result"
```

---

## Întrebarea 153

Q:

8. Ce va afișa în consolă fragmentul de cod de mai jos?

```
char* s[3] = {"A", "B", "C"};  
for(i=0; i<3; i++) {  
    execl("/bin/echo", "/bin/echo", s[i], NULL);  
}
```

A:

Va afișa doar "A", deoarece după execuția lui execl codul procesului este suprascris cu noul cod generat de execl. Familia exec() înlocuiește procesul curent cu unul nou. Bucla nu va ajunge niciodată la "B" sau "C".

---

## Întrebarea 154

**Q:**

9. Ce face apelul de sistem "read" când FIFO-ul conține mai puține date decât se cere să citească?

**A:**

Va citi datele existente din FIFO.

---

## Întrebarea 155

**Q:**

10. Ce va afișa codul de mai jos, dacă niciun alt proces nu deschide FIFO-ul "abc"? Justificați răspunsul.

```
int r, w, n = 0;
r = open("abc", O_RDONLY);
n++;
w = open("abc", O_WRONLY);
n++;
printf("%d\n", n);
```

**A:**

Codul nu va afișa nimic, deoarece va rămâne blocat la primul open, așteptând ca un alt proces să deschidă FIFO-ul la capătul de scriere. După deschiderea FIFO-ului pentru citire, programul va aștepta ca celălalt proces să deschidă FIFO-ul pentru scriere.

---

## Întrebarea 156

**Q:**

11. Ce se întâmplă cu un proces între momentul în care se termină și momentul în care părintele apelează wait?

**A:**

Va intra într-o stare de zombie, neexecutând cod dar ocupând un PID. Când rulezi "ps -ef", va apărea ca <defunct>.

Între momentul în care procesul se termină și momentul în care părintele apelează wait, procesul devine zombie.

---

## Întrebarea 157

**Q:**

17. Câte blocuri de date pot fi referențiate prin indirectarea dublă a unui i-node, dacă un bloc conține N adrese către alte blocuri?

**A:**

$N \times N = N^2$  blocuri de date.

Explicație: indirectarea dublă funcționează astfel:

- l-node-ul are un pointer spre un bloc de indirectare dublă
- Acel bloc conține N adrese (pointeri) spre alte blocuri
- Fiecare dintre acele N blocuri conține N adrese către blocuri de date
- Total:  $N \times N = N^2$  blocuri de date

---

## Întrebarea 158

Q:

20. Adăugați instrucțiunile necesare la fragmentul de cod de mai jos, astfel încât stdin-ul comenzii /bin/pwd să fie citit din PIPE-ul p.

```
int p[2];
pipe(p);
if(fork() == 0) {
    execl("/bin/pwd", "/bin/pwd", NULL);
    exit(0);
}
```

A:

```
int p[2];
pipe(p);
if (fork() == 0) {
    close(p[1]);          // închidem capătul de scriere în copil
    dup2(p[0], 0);        // redirectăm capătul de citire la stdin
    execl("/bin/pwd", "/bin/pwd", NULL);
    exit(0);
}
close(p[0]);             // în părinte închidem capătul de citire
// scriem în p[1] datele pentru pwd, apoi close(p[1])
```

---

## Întrebarea 159

Q:

1.

Scrieti o comanda UNIX care afiseaza toate liniile din fisierul a.txt care contin cel putin un num

ar binar

multiplu de 4 cu cinci sau mai multe cifre (ex: 010100).

A:

```
grep -E '[0-1]{3,}[0]{2}' a.txt
```

Explicatie: ultimele 2 cifre din fiecare nr binar trebuie sa fie 0 (ele reprezinta suma cu 2 si 1, daca ele sunt 1 atunci nu e multiplu

de 4). In rest, cifrele din stanga pot fi in numar de 3 sau mai mare decat 3 (ca zice cu cinci sau mai multe cifre).

---

## Întrebarea 160

Q:

2.

Scrieti o comanda UNIX care inverseaza toate perechile de cifra impara urmata de vocala

A:

echo a23e8i97u3 | sed -E 's/([13579])([aeiou])\2\1/gi'

Explicatie: primul caracter sa fie oricare din lista [13579], al doilea oricare din lista vocal

elor, dupa dai swap global netinand

cont de majuscule

---

## Întrebarea 161

Q:

3. Scrieti o comanda UNIX care afiseaza toate scorurile de fotbal unice (ex: 4-0) care apar in fisierul a.tx

t.

Numarul de goluri poate avea maximum doua cifre

A:

cat a2.txt | sed 's/ /\n/g' | uniq -u | grep -E '^[0-9]{1,2}-[0-9]{1,2}\$'

Explicatia: inlocuiesti spatiul cu enter, ca sa ai fiecare scor pe o linie. folosesti unig -u ca sa obtii doar liniile unice , apoi grep pentru a respecta conditiile.

---

## Întrebarea 162

Q:

4

. Afisati numarul de procese ale fiecarui utilizator activ din sistem

A:

Comandă:

```
who | awk '{print $1}' | sort | uniq | awk '{print "ps -u \"$1\" | wc -l"}'  
| /bin/sh
```

Explicație: who pentru a afla utilizatorii activi, sort și uniq ca să-i ai o singură dată. La awk pui delimitatorul să fie spațiu pentru o afișare frumoasă, apoi printezi utilizatorul și apoi dai comanda system ca să afli câte procese are.

---

## Întrebarea 163

Q:

5

. Scrieti un script Shell UNIX care calculeaza media de fisiere cu extensia .txt per director din directorul

curent si toate subdirectoarele lui.

A:

```
#!/bin/bash
```

```
nr_txt=0
```

```
nr_dir=0
```

```
for f in $(find $(pwd) -type f); do  
    if [[ "$f" == *.txt ]]; then  
        echo "$f este fisier .txt"  
        nr_txt=$((nr_txt + 1))  
    fi  
done
```

```
for d in $(find $(pwd) -type d); do  
    echo "$d este director"  
    nr_dir=$((nr_dir + 1))  
done
```

```
echo "Sunt $nr_txt fisiere text"
```

```
echo "Sunt $nr_dir directoare"
```

---



## Întrebarea 164

Q:

6

. Cate procese va crea fragmentul de cod de mai jos, excluzand procesul parinte initial ?

```
f(fork()!=fork()){  
  
fork();
```

A:

6 copii (7 procese inclusiv părintele).

Execuție pas cu pas:

- P face primul fork() → C1. În P returnează PID(C1), în C1 returnează 0.
- P face al doilea fork() → C2.
- C1 face al doilea fork() → C3.
- Acum se evaluează condiția fork() != fork():
  - În P: PID(C1) != PID(C2) → true → P face fork() → C4
  - În C1: 0 != PID(C3) → true → C1 face fork() → C5
  - În C2: PID(C1) != 0 → true → C2 face fork() → C6
  - În C3: 0 != 0 → false → NU face fork

Copii creați: C1, C2, C3, C4, C5, C6 = 6 procese.

---

## Întrebarea 165

Q:

Ce tipărește în consolă fragmentul de cod de mai jos?

```
char* s[3] = {"A", "B", "C"};  
for(i=0; i<3; i++) {  
    if(fork() != 0) {  
        execl("/bin/echo", "/bin/echo", s[i], NULL);  
    }  
}
```

A:

Afișează "A", "B", "C" – fiecare pe o linie nouă, în ordine NEDETERMINATĂ.

Analiza:

- La fiecare iterație, fork() creează un copil. Doar PĂRINTELE (fork != 0) intră în if și execută echo. Copilul (fork == 0) NU intră în if și continuă bucla.
- i=0: P fork → C1. P execută echo "A" și se înlocuiește (se termină după). Copilul C1 continuă bucla.
- i=1, în C1: fork → C2. C1 execută echo "B" și se înlocuiește. C2 continuă.
- i=2, în C2: fork → C3. C2 execută echo "C". C3 continuă, iese din buclă și se termină.

P, C1, C2 rulează echo în paralel – ordinea în care apar A, B, C pe ecran depinde de planificator (poate fi BAC, CAB, ABC etc.).

---

## Întrebarea 166

Q:

9

.

Ce face apelul sistem "write" cand in PIPE este spatiu, dar nu suficient pentru cat i se cere sa scrie?

A:

Uneori un write reușit poate transfera mai puțini octeți decât numărul cerut (count). Astfel de scrieri parțiale pot apărea în diferite contexte, unul dintre ele fiind cel de mai sus.

Scrie câți octeți sunt liberi și returnează câți octeți au fost scriși corect.

---

## Întrebarea 167

Q:

10

. Ce tiparește fragmentul de cod de mai jos daca niciun alt proces nu deschide FIFO-

ul "abc"?

Justificati raspunsul

```
= open("abc", O_WRONLY);  
n=write(r, &k, sizeof(int));  
printf("3éd\n", n);
```

A:

Fragmentul NU va afișa nimic, pentru că procesul se blochează la apelul open().

Explicație: open(path, O\_WRONLY) pe un FIFO se blochează până când un alt proces deschide același FIFO pentru citire (O\_RDONLY). Dacă niciun alt proces nu deschide FIFO-ul pentru citire, procesul rămâne blocat la open() la nesfârșit.

Deci write() și printf() de mai jos nu se execută niciodată.

---

## Întrebarea 168

Q:

12

. Considerati ca functia f este executata simultan de 10 thread-uri. Adaugati liniile de cod

necesare ca

sa asigurati ca n va avea valoarea 10 dupa ce thread-urile isi incheie executia ?

```
int n=0;
pthread_mutex_t mtx=PTHREAD_MUTEX_INITIALIZER;
```

```
void* f(void* p) {
pthread_mutex_lock(&mtx);
nt
pthread_mutex_unlock(&mtx);
return NULL;
```

A:

Trebuie adăugat un mutex care protejează operația n++.

Înainte de funcția f:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
```

În funcția f, în jurul lui n++:

```
pthread_mutex_lock(&mtx);
n++;
pthread_mutex_unlock(&mtx);
```

La sfârșit (în main, după pthread\_join):

```
pthread_mutex_destroy(&mtx);
```

De ce e necesar? Operația n++ NU e atomică – în spate are 3 pași: citire n din memorie, adunare 1, scriere înapoi. Fără mutex, două thread-uri pot citi aceeași valoare, o incrementează și o scriu – o incrementare se pierde (race condition). Cu mutex, doar un thread la un moment dat face n++.

---

## Întrebarea 169

Q:

13. Planificati executia job-urilor urmatoare (date ca Nume/Durata/Termen) incat suma intarzieri lor job-

urilor sa fie minima: A/22/27, B/2/15, C/4/5

A:

Ordinea optimă: C, B, A — întârziere totală = 1

Joburi:

- A: durată 22, deadline 27
- B: durată 2, deadline 15
- C: durată 4, deadline 5

Verificare ordine C, B, A:

- C: pornește la 0, termină la 4, deadline 5 → întârziere 0
- B: pornește la 4, termină la 6, deadline 15 → întârziere 0
- A: pornește la 6, termină la 28, deadline 27 → întârziere 1

Total:  $0 + 0 + 1 = 1$

E optim — în orice altă ordine, întârzierea totală e mai mare.

---

## Întrebarea 170

Q:

15. Care este cea mai prioritara categorie de pagini de memorie din care politica de inlocuire N

RU ar

alege o pagina victima

A:

Clasa O (0,0): pagini ne-referențiate și ne-modificate.

NRU selectează pagina care aparține primei clase (clasa 0) care nu este goală. Dacă nu există pagini în clasa 0, selectează prima pagină din clasa 1, și așa mai departe.

Clasele:

- Clasa 0: Ne-referențiate, ne-modificate ( $r=0, w=0$ ) — prioritate maximă pentru evacuare
- Clasa 1: Ne-referențiate, modificate ( $r=0, w=1$ )
- Clasa 2: Referențiate, ne-modificate ( $r=1, w=0$ )
- Clasa 3: Referențiate, modificate ( $r=1, w=1$ )

Deci pagina cea mai prioritară pentru a fi aleasă ca victimă este cea cu biții  $r$  și  $w$  setați la 0.

---

## Întrebarea 171

Q:

17.

Dandu-se 2 cache-uri set-asociative, unul cu 2 seturi de 4pagini si unul cu 4setu

ri de 2pagini, care va

da rezultatele mai bune pt secventa de cereri de pagini: 14,23,1,16,1,23,16,14. Justificati raspu

nsul.

A:

4seturi de 2 pagini.

$$14\%4 = 2$$

23

-> 3

1 -> 1

16

-> 0

1 -> 1

23

-> 3

16

-> 0

$$14\%4 = 2$$

pt 4seturi de 2 pagini. Gr: 0

—

16, 1-1, 2-14, 3-

23

pt 2seturi de 4 pagini . Gr: 0->14, 16 .. 1->23, 1

Depinde de cum merge:

---

## Întrebarea 172

Q:

18. Câte blocuri de date pot fi referite prin tripla-indirectare a unui i-node, dacă un bloc are dimensiunea B și o adresă are dimensiunea A?

A:

$(B/A)^3$  blocuri de date.

Explicație: un bloc conține B/A adrese. Indirectarea triplă are 3 niveluri:

- Nivel 1: pointer spre un bloc cu B/A adrese
- Nivel 2: fiecare adresă spre un bloc cu B/A adrese
- Nivel 3: fiecare adresă spre un bloc cu B/A adrese
- Nivel 4: fiecare adresă spre un bloc de date

Total blocuri de date:  $(B/A) \times (B/A) \times (B/A) = (B/A)^3$

---

## Întrebarea 173

Q:

19. Ce se întâmplă cu un link hard când fișierul spre care punctează este șters?

A:

Păstrează datele fișierului șters chiar dacă fișierul nu mai există.

Când creezi un hard link către un fișier text și apoi ștergi fișierul text, nu se întâmplă nimic cu hard link-ul - acesta păstrează datele de la fișierul șters.

Dacă ștergi hard link-ul, fișierul original NU este șters (mai există atât timp cât cel puțin un hard link mai există).

(Un fișier = inode care indică spre un bloc de date.)

Pentru un hard link, i-node-urile indică spre date, deci când ștergem un fișier, datele rămân (atât timp cât există încă referințe la inode).

---

## Întrebarea 174

Q:

1. Scrieți o comandă UNIX Shell care afișează liniile dintr-un fișier a.txt care conțin cuvinte ce încep cu literă mare.

A:

```
grep -E "^[A-Z]" a.txt
```

```
grep -E "\<[A-Z].*\>" a.txt
```

---

## Întrebarea 175

Q:

2. Scrieți o comandă UNIX Shell care inversează în fișierul a.txt toate perechile de cifre vecine.

A:

```
sed -E "s/([0-9])([0-9])/^2\1/gi" a.txt
```

---

## Întrebarea 176

Q:

3. Fișierul a.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor.

A:

```
awk '{print s=$1 + $2 }' number
```

---

## Întrebarea 177

Q:

Afișați doar liniile dintr-un fișier a.txt care apar o singură dată (nu sunt duplicate).

A:

```
sort a.txt | uniq -u
```

Explicație:

- sort sortează liniile (necesar pentru uniq, care lucrează pe linii consecutive)
- uniq -u afișează doar liniile UNICE (-u = unique), adică cele care apar exact o singură dată

Fără -u, uniq afișează prima apariție din fiecare grup. Cu -u, afișează doar grupurile cu o singură linie.

---

## Întrebarea 178

Q:

5. Scrieți un script UNIX Shell care afișează numele fiecărui fișier .txt din directorul curent care conține cuvântul "cat".

A:

```
tre WOrg dat
```

```
files="find ./-maxdepth 1 type f
```

```
ok=0
```

```
for file in $files
```

```
do
```

```
IFS=$'\n'
```

```
ok=0
```

```
for line in $(cat $file)
```

```
do
```

```
if echo $line | grep -E -q "cat"
```

```
then
```

```
ok=1
```

```
fi
```

```
done
```



if [Sok-eq1]

then

echo Sfile

fi

OO A&A Page

done

---

## Întrebarea 179

Q:

*(fără conținut)*

A:

Cazul 1: (execl reușește)

→ Output-ul va fi "A", deoarece comanda execl șterge codul inițial și îl înlocuiește cu codul noului program.

Cazul 2: (execl nu reușește)

→ Output-ul va fi "A" și apoi "B", deoarece apelul execl a eșuat și procesul apelant continuă să execute.

---

## Întrebarea 180

Q:

14. Dați un avantaj și un dezavantaj al alocării segmentate față de alocarea paginată.

A:

Dezavantaj: Alocarea segmentată este mai lentă decât cea paginată în privința accesului la memorie.

Avantaj: Alocarea segmentată oferă o protecție mai bună a memoriei, deoarece fiecare segment poate primi drepturi de acces diferite.

---

## Întrebarea 181

Q:

20. Ce este un semafor binar și care este efectul metodei P când este apelată de mai multe procese/thread-uri concurente?

A:

Un semafor binar este restricționat la valorile 0 și 1, putând fi folosit pentru a controla accesul la o singură resursă, în special pentru a asigura excludere mutuală pentru o secțiune critică (este cunoscut și ca mutex lock).

Metoda P (= wait, sleep, down): dacă semaforul este deblocat (valoare = 1), devine blocat (valoare = 0) și apelul se întoarce, permițând thread-ului să continue. Dacă semaforul este blocat (valoare = 0), thread-ul este blocat și plasat într-o coadă Q de thread-uri în așteptare.

Semaforul este inițializat cu 1. Folosește două operații atomice: 1) wait (P), 2) signal (V) pentru sincronizarea proceselor.

---

## Întrebarea 182

**Q:**

Avantaj si dezavantaj al alocarii cu partitiile variabile si alocarea paginata

**A:**

Avantaj alocare cu partiții variabile (față de paginată):

- Adresa nu trebuie translatată dintr-o adresă virtuală într-una fizică (sau translatarea este foarte simplă), deci accesul la memorie este mai rapid din punct de vedere computațional.
- Nu există fragmentare internă (procesul primește exact cât are nevoie).

Dezavantaj:

- Apare fragmentarea EXTERNĂ: după multe alocări și eliberări, memoria rămâne plină de spații libere mici, neutilizabile.
- Compactarea memoriei (pentru a uni spațiile libere) este costisitoare și nu se poate face online ușor.
- Necesită ca un proces să fie încărcat în memorie continuă (de la o adresă până la alta neîntreruptă), ceea ce e o restricție grea.

---

## Întrebarea 183

**Q:**

Cand trece un process din starea de RUN in starea de READY

De ce operatiile I/O cauzeaza un Proces: 'treaca din starea de RUN in

Meeteetse cee etiors couse a process to mons ee 'the

Cum se ore calculul de adresa in alocarea cu partis:

ee nl

**A:**

Un proces își va schimba starea din RUN în READY după ce a făcut tot ce avea de făcut în timpul alocat pe procesor (după ce expiră cuanta sa de timp pe procesor).

De asemenea, dacă procesul face o operație I/O, va trece din RUN în WAIT (nu READY), deoarece operațiile I/O pot dura mult, iar scheduler-ul rulează alt proces care e în Ready (scheduler-ul este preemptiv, anticipativ).

---

## Întrebarea 184

**Q:**

*(fără conținut)*

**A:**

*(fără conținut)*

---

## Întrebarea 185

**Q:**

Implementati programele A, B si C care comunica prin FIFO. The 5 ori, A si B (jucatori) ii trimit lui C (loteria) cate 6 intregi de la 1 la 49 pe care C ii compara cu 6 intregi intre 1 si 49 alesi la pornire. Loteria raspunde jucatorilor x cu numarul de intregi ghiciti. Jucatorii afiseaza de fiecare data cati intregi

au ghicit.

**A:**

```
// Loteria (proces C):
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <time.h>

#define N 5

int main() {
    mkfifo("/tmp/a2c", 0666);
    mkfifo("/tmp/b2c", 0666);
    mkfifo("/tmp/c2a", 0666);
    mkfifo("/tmp/c2b", 0666);

    srand(time(NULL));
    int extrase[6];
    for(int i = 0; i < 6; i++) extrase[i] = rand() % 49 + 1;

    int fa = open("/tmp/a2c", O_RDONLY);
```

```

int fb = open("/tmp/b2c", O_RDONLY);
int wa = open("/tmp/c2a", O_WRONLY);
int wb = open("/tmp/c2b", O_WRONLY);

for(int r = 0; r < N; r++) {
    int ja[6], jb[6];
    read(fa, ja, sizeof(ja));
    read(fb, jb, sizeof(jb));

    int ga = 0, gb = 0;
    for(int i = 0; i < 6; i++)
        for(int j = 0; j < 6; j++) {
            if(ja[i] == extrase[j]) ga++;
            if(jb[i] == extrase[j]) gb++;
        }
    write(wa, &ga, sizeof(int));
    write(wb, &gb, sizeof(int));
}
close(fa); close(fb); close(wa); close(wb);
unlink("/tmp/a2c"); unlink("/tmp/b2c");
unlink("/tmp/c2a"); unlink("/tmp/c2b");
return 0;
}

// Jucător (A sau B) – schimbă numele FIFO-urilor:
int main() {
    int w = open("/tmp/a2c", O_WRONLY);
    int r = open("/tmp/c2a", O_RDONLY);
    srand(time(NULL));
    for(int i = 0; i < 5; i++) {
        int nums[6];
        for(int j = 0; j < 6; j++) nums[j] = rand() % 49 + 1;
        write(w, nums, sizeof(nums));
        int ghicit;
        read(r, &ghicit, sizeof(int));
        printf("A ghicit: %d\n", ghicit);
    }
    close(w); close(r);
    return 0;
}

```

---

## Întrebarea 186

**Q:**

Scrieti o expresie regulata care accepta orice lant care contin cel putin tret vocale.

**A:**

Grep -E '([aeiou].\*){3,}' fisier

---

## Întrebarea 187

**Q:**

Ce va afisa in consola fragmentul de cod de mat jos? Justificati raspunsul.

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {

execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

Raspundeti mai jos/Write your answer below:

**A:**

Va afișa doar "A".

Explicație: la prima iterație (i=0), execl("/bin/echo", "/bin/echo", s[0], NULL) reușește. Familia exec() ÎNLOCUIEȘTE procesul curent cu un alt program. Astfel, procesul devine /bin/echo, care afișează "A" și se termină.

Codul de DUPĂ execl (restul iterațiilor buclei for) NU se mai execută niciodată – programul original a fost suprascris.

Pentru a afișa A, B, C ar fi trebuit fork pentru fiecare iterație:

```
if (fork() == 0) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
    exit(1);
}
wait(NULL);
```

---

## Întrebarea 188

**Q:**

Care este principiul vecinatatit in privinta incarcarii paginilor unui proces?

**A:**

Cand incarcam o pagina refetita, incarcam si cateva pagini de langa ea

---

## Întrebarea 189

**Q:**

Plantificati Job-urile de mat jos (date ca Nume/Durata/Termen) incat suma intarzierilor task-urilor sa fie minima.

**A:**

BADC

24+54+3+4

O intarziere de 2 la D si una la C. intarziere totala de 3

---

## Întrebarea 190

Q:

Ce risc ridtca functta f daca este rulata tn mat multe thread-urt stnultane?  
Justificatt raspunsul.

```
/* R0: Consideratt ca mutex-it sunt initialtzatt corect */  
Consider that the mutexes are properly tnittaltzed */  
1 mutex_t m[2];  
votd* f(votd* p) {  
  
    tnt td = (int)p;  
  
    pthread_mutex_t* first = am[td x 2];  
  
    pthread_mutex t* second = am[(tde1) % 2];  
  
    pthread_nutex_lock(first);  
    pthread_mutex_tock(second) ;  
  
    pthread_mutex_unlock(second);  
    pthread_mutex_unlock(first);  
  
}
```

Raspundett mat jos/Write your answer below:

A:

Deadlock

Lock(0) lock(1)

Lock(1) lock(0)

Si apuca fiecare thread sa faca primul lock.

---

## Întrebarea 191

Q:

Dati un avantaj si un dezavantaj al politicii de plasare First-Fit fata de Worst-Fit.

A:

First Fit vs Worst Fit:

First Fit (primul potrivit):

- Parcurge lista de blocuri libere și folosește PRIMUL bloc suficient de mare
- Avantaj: rapid (nu parcurge toată lista)
- Dezavantaj: lasă multe fragmente mici (fragmentare externă) la începutul memoriei

Worst Fit (cel mai mare):

- Folosește CEL MAI MARE bloc liber disponibil
- Avantaj: după alocare rămâne un fragment mai mare, util pentru alocări viitoare
- Dezavantaj: parcurge mereu toată lista (mai lent); poate consuma blocurile mari prematur

În practică, First Fit este preferat pentru viteza sa.

---

## Întrebarea 192

Q:

Care este cea mai prioritara pagina de memorie pe care politica de inlocuire NRU ar alege-o ca pagina victima?

A:

Politica NRU (Not Recently Used) alege pagina victimă din prima clasă nevidă, în ordinea:

- Clasa 0: Ne-referențiată, ne-modificată (R=0, M=0) — CEA MAI PRIORITARĂ pentru eliminare
- Clasa 1: Ne-referențiată, modificată (R=0, M=1)
- Clasa 2: Referențiată, ne-modificată (R=1, M=0)
- Clasa 3: Referențiată, modificată (R=1, M=1) — ultima alegere

Deci pagina cea mai prioritară de evacuat este una din Clasa 0 (R=0, M=0). Sunt pagini care nu au fost folosite recent și nu trebuie scrise înapoi pe disc (modificate).

---

## Întrebarea 193

Q:

(fără conținut)

A:

$(B/A)^3$

---

## Întrebarea 194

Q:

(fără conținut)

A:

Grep -E -I '[a]{1,}' fisier | grep -E -I -v '[b]{1,}' fisier

Grep -E -I '^[^b]\*a[^b]\*\$' fisier

---

## Întrebarea 195

**Q:**

Care este numărul maxim de procese fiu create de fragmentul de cod de mai jos care pot coexista simultan?

```
for(i=0; i<7; i++) {  
    if(fork() == 0) {  
        sleep(rand() % 10);  
        exit(0);  
    }  
    if(i % 3 == 0) {  
        wait(0);  
    }  
}
```

**A:**

5 copii pot coexista simultan.

Analiza pas cu pas pentru părinte:

- i=0: fork → C1 (1 copil).  $i\%3==0$  → wait → așteaptă C1 (0 copii).
- i=1: fork → C2 (1 copil).  $i\%3\neq0$  → nu așteaptă.
- i=2: fork → C3 (2 copii).  $i\%3\neq0$  → nu așteaptă.
- i=3: fork → C4 (3 copii).  $i\%3==0$  → wait → așteaptă un copil să se termine (2 copii).
- i=4: fork → C5 (3 copii). nu așteaptă.
- i=5: fork → C6 (4 copii). nu așteaptă.
- i=6: fork → C7 (5 copii).  $i\%3\neq0$  → nu așteaptă.

Maxim simultan: 5 copii.

(Numărul exact depinde de ordinea de execuție, dar 5 este maximul teoretic atins după ultimul fork.)

---

## Întrebarea 196

**Q:**

*(fără conținut)*

**A:**

3. Dacă de exemplu am 4 core-uri aș porni de la 4 thread-uri și așa știu că lucrează în paralel pe core-uri diferite. Măsoară timpul de execuție. Și mă duc cu multiplii de 4 până când timpul de rulare crește și se alungește numărul de thread-uri pentru care am obținut cel mai mic timp de execuție

---



## Întrebarea 197

Q:

(fără conținut)

A:

4. Pentru ca dureaza mult si ca sa nu tina ocupat procesorul le punem in wait pana se incarca

---

## Întrebarea 198

Q:

(fără conținut)

A:

5. Adresa executabilului e aceeași cu adresa fizică.

---

## Întrebarea 199

Q:

dar nu conține litera

A:

a tn ay

(grep -E) "A[Ab]+a\*[~b]+\$"

Uhome/xm/exam/os/account/s,

---

## Întrebarea 200

Q:

'xm/exam/os/account/odir2783/ex2419> cat wememlp.txt

ți un avantaj și un dezavantaj al încărcării tuturor paginilor unui program la pornirea procesului în comparație cu încărcarea lor doar când sunt necesare.

77

A:

Vezi răspunsul de la întrebarea 94.

Încărcarea TUTUROR paginilor la pornire:

+ Avantaj: nu apar page faults în timpul execuției → execuție fluidă

- Dezavantaj: irosim memorie (încărcăm și pagini ce nu vor fi folosite);  
pornirea e lentă

Încărcarea LA CERERE (demand paging):

- + Avantaj: folosim eficient memoria, pornire rapidă
- Dezavantaj: page faults în timpul execuției → pauze

---

## Întrebarea 201

Q:

Ce va afișa fragmentul de cod de mai jos, considerând că thread-urile se creează fără probleme? Justificați răspunsul.

```
/* Considerați că header-ele necesare sunt incluse aici */
void* f(void* a) {
    printf("%d\n", *(int*)a);
    return NULL;
}

int main() {
    /* Considerați că variabilele necesare sunt declarate aici */
    for(i=0; i<10; i++) {
        pthread_create(&t[i], NULL, f, &i);
    }
    /* Considerați că aici se fac join-urile necesare */
    return 0;
}
```

Răspundeți mai jos:

A:

Output: nu este cert ce vor afișa thread-urile.

Vor fi afișate 10 numere, dar valorile și ordinea sunt impredictibile, pentru că:

- Thread-urile primesc adresa lui i (pointer), nu valoarea
- Variabila i este incrementată de bucla principală în paralel cu thread-urile
- Când un thread accesează \*(int\*)a, citește valoarea curentă a lui i, care poate fi orice între 0 și 10

Posibile output-uri: "0 1 2 3 4 5 6 7 8 9" (ideal), dar mai probabil ceva ca "3 3 5 7 7 9 10 10 10 10".

Soluție corectă: trimite COPIE a lui i, nu pointer:

```
int *p = malloc(sizeof(int)); *p = i; pthread_create(&t, NULL, f, p);
```

---

## Întrebarea 202

Q:

Care sunt elementele unei adrese virtuale în alocarea paginat-segmentată și ce tabele sunt implicate în calcularea adresei fizice?

**A:**

Adresa virtuală în alocarea paginat-segmentată: (s, p, d)

- s = numărul segmentului
- p = numărul paginii virtuale în segment
- d = deplasamentul (offset) în pagină

Tabelele implicate:

1. Tabela de segmente a procesului — pentru fiecare segment indică o tabelă de pagini
2. Tabela de pagini (una pentru fiecare segment) — pentru fiecare pagină indică numărul cadrului fizic

Calcul: s → tabela de segmente → tabela de pagini a segmentului → p → numărul cadrului fizic  
→ + d → adresa fizică.

---

## Întrebarea 203

**Q:**

Adăugați codul sursă necesar la fragmentul de cod de mai jos pentru ca printf să afișeze în consolă.

```
int p[2];
pipe(p);
dup2(p[1], 1);
printf("asdf\n");
```

Răspundeți mai jos:

**A:**

```
#include <stdio.h>
#include <unistd.h>
```

```
int main() {
    int p[2];
    pipe(p);

    if (fork() == 0) {
        // Proces copil: citește din pipe și afișează
        close(p[1]);
        dup2(p[0], 0); // redirectează stdin la pipe
        char buf[256];
        fgets(buf, sizeof(buf), stdin);
        printf("Copilul a citit: %s", buf);
        close(p[0]);
    } else {
        // Părintele: scrie în pipe
        close(p[0]);
        dup2(p[1], 1); // redirectează stdout la pipe
        printf("asdf\n");
    }
}
```

```
        fflush(stdout);
        close(p[1]);
    }
    return 0;
}
```

---

## Întrebarea 204

**Q:**

Ce va afisa in consola fragmentul de cod de mai jos? Justificati raspunsul.

```
char* s[3] = ("A", "B", "C");
for(i=0; i<3; i++) {

    execl("/bin/echo", "/bin/echo", s[i], NULL);
    t
```

Raspundeti mai jos/Write your answer below:

**A:**

Va afișa doar "A".

De ce? La prima iterație, `execl("/bin/echo", "/bin/echo", "A", NULL)` reușește și înlocuiește procesul curent cu `/bin/echo`, care afișează "A" și se termină.

Familia `exec()` înlocuiește procesul curent cu un alt program. Codul de DUPĂ `exec` (restul iterațiilor buclei `for`) nu se mai execută niciodată – programul original a fost suprascris.

Pentru a afișa A, B, C, ar trebui `fork` pentru fiecare iterație:

```
if (fork() == 0) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
    exit(1);
}
wait(NULL);
```

---

## Întrebarea 205

**Q:**

Scrieti o expresie regulara care accepta orice linti care contin cel putin trei vocale.

**A:**

```
^[^aeiou]*[aeiou][^aeiou]*[aeiou][^aeiou]*[aeiou].*$
```

Sau folosind lookahead-uri (PCRE):

```
^(?=(.*[aeiou]){3}).*$
```

Varianta mai simplă cu `grep`:

```
grep -E '([aeiou].*){3}' fisier
```

Explicație:

- [aeiou] = o vocală
- .\* între ele = orice caractere (inclusiv nimic)
- de 3 ori = cel puțin 3 vocale

---

## Întrebarea 206

Q:

Care este principiul vecinatatii in privinta incarcarii paginilor unui proces?

A:

Principiul vecinătății (sau principiul localității):

Un proces va avea probabil nevoie în curând de paginile vecine paginii tocmai accesate. Această observație are două forme:

- Localitate spațială: dacă o adresă este accesată, adresele vecine au șanse mari să fie accesate în curând
- Localitate temporală: o adresă accesată recent are șanse să fie accesată din nou

Aplicare: când încărcăm o pagină în memorie, "preîncărcăm" și vecinii ei (prefetching). Asta reduce numărul de page faults.

---

## Întrebarea 207

Q:

4. Dat fiind un director și ierarhia sa, scrieți o comandă UNIX Shell care afișează toate albumele din el. Un album este un director care conține fișiere cu extensia ".mp3".

A:

```
#!/bin/bash
find . -name "*.mp3" -printf "%h\n" | sort -u
```

Sau, dacă vrem o variantă explicită:

```
#!/bin/bash
for d in $(find . -type d); do
    if ls "$d"/*.mp3 2>/dev/null | grep -q .; then
        echo "$d este un album"
    fi
done
```

Explicație: find -printf "%h\n" afișează doar calea directorului care conține fișierul găsit. sort -u elimină duplicatele.

---

## Întrebarea 208

Q:

3. Display a list of all unique appearances of number Pi with 2 or more decimals in a.txt.

A:

```
grep -E-i-o "(3\\.14[0-9]*)" a.txt | sort | uniq
```

---

## Întrebarea 209

Q:

2. Display lines of file a.txt by swaping any pairs of vowel followed by odd digit (a23b7a98i3 -> a23b79a83i).

A:

```
sed -E "s/([aeiou])([13579])\2\1/gi" a.txt
```

---

## Întrebarea 210

Q:

1. Display lines in a.txt that contain at least 1 timestamp (hh:mm:ss:sss).

A:

```
grep -E -i "^.*((([0-9]{2}:){3}[0-9]{3}).*)" a.txt
```

---

## Întrebarea 211

Q:

5. Scrieți un script UNIX Shell care calculează numărul mediu de linii de cod în toate fișierele cu extensia ".sh" din directorul curent, ignorând liniile de comentariu și liniile care conțin doar spații și taburi.

A:

```
#!/bin/bash
```

```
total=0
```

```
nr=0
```

```
for f in *.sh; do
```

```
    if [ -f "$f" ]; then
```

```
        # Numărăm liniile care NU sunt comentarii sau goale
```

```
        linii=$(grep -vE '^\s*(#|$)' "$f" | wc -l)
```

```
        total=$((total + linii))
```

```
        nr=$((nr + 1))
```

```
    fi
```

```
done
```

```
if [ $nr -gt 0 ]; then
```

```
    echo "Media: $(echo "scale=2; $total / $nr" | bc)"
```

```
else
    echo "Nu există fișiere .sh"
fi
```

Explicație: `grep -vE '^\s*(#|$)'` filtrează liniile care încep cu # (comentarii) sau sunt goale/cu spații.

---

## Întrebarea 212

**Q:**

Câte procese sunt create?

```
if(fork() || fork()) {
    fork();
}
```

**A:**

3 procese sunt create (excluzând părintele).

Analiza:

- `fork()` #1 → creează C1. În părinte returnează PID-ul lui C1 ( $\neq 0$ , deci `truthy`). În C1 returnează 0 (`falsy`).
- În părinte: condiția primului `fork()` e `truthy`, deci `||` NU evaluează al doilea `fork`; intră în `if` și face un nou `fork()` → creează C2.
- În C1: primul `fork()` returnează 0 (`falsy`), deci `||` EVALUEAZĂ al doilea `fork()` → creează C3. Apoi în C1 condiția devine `PID(C3)` (`truthy`), intră în `if` și creează încă un `fork` → C4. În C3, ambele sunt 0, deci condiția e falsă, nu face `if`.

Total: C1, C2, C3, C4 = 4 procese.

Observație: răspunsul exact depinde de interpretarea exactă a operatorului `||` cu `fork()` – recomandat să verifici cu un program de test.

---

## Întrebarea 213

**Q:**

7. Câte procese sunt create când procesul părinte apelează `f(3)`?

```
void f(int n) {
    if (n > 0 || fork() == 0) {
        f(n-1);
        exit(0);
    }
    wait(0);
}
```

**A:**

Numărul de procese create teoretic este INFINIT (sau limitat doar de resursele sistemului).

Analiza codului `f(n)`:

- Dacă  $n > 0$ : condiția e true (datorită `||`), apelează `f(n-1)`, apoi exit.
- Dacă  $n \leq 0$ : condiția devine `fork() == 0`:
  - În părinte: `fork()` returnează `PID > 0` → false → execută `wait(0)`
  - În copil: `fork()` returnează `0` → true → apelează `f(n-1)` → ... și TOT FACE FORK la fiecare nivel.

Deci pentru `f(3)`:

- `f(3) → f(2) → f(1) → f(0)` → în `f(0)` fork creează un copil.
- Copilul apelează `f(-1)` → în `f(-1)` face din nou fork → ... la INFINIT.

În practică, sistemul rămâne fără resurse, `fork()` returnează `-1` ( $\neq 0$ ), condiția devine false și se ajunge la wait.

---

## Întrebarea 214

Q:

Ce va afișa codul de mai jos?

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

A:

Va afișa DOAR "A".

Explicație: la prima iterație ( $i=0$ ), `execl("/bin/echo", "/bin/echo", "A", NULL)` reușește. Familia `exec()` ÎNLOCUIEȘTE procesul curent cu programul "echo", care afișează "A" și se termină.

Codul de DUPĂ `execl` (restul iterațiilor buclei `for`) NU se mai execută niciodată – programul original a fost suprascris.

Observație: dacă întrebarea ar fi avut `fork()` înainte de `execl`, atunci ar fi afișat A, B, C în ordine aleatorie. Fără `fork`, doar prima iterație rulează.

---

## Întrebarea 215

Q:

9. Ce face apelul de sistem "read" când pipe-ul conține mai puține date decât se cere să citească, dar nu este gol?

A:

`read()` va citi cât poate — adică toate datele disponibile în pipe, chiar dacă sunt mai puține decât a cerut. Returnează numărul efectiv de octeți citați.

Exemplu: dacă cer `read(fd, buf, 100)` dar în pipe sunt doar 30 octeți, `read` va întoarce 30 (cei disponibili) imediat.



DIFERENȚA față de cazul "pipe gol":

- Pipe gol și writer activ: read() BLOCHEAZĂ
  - Pipe gol și writer închis: read() returnează 0 (EOF)
  - Pipe cu date (chiar dacă mai puține decât cerute): read() returnează imediat ce poate
- 

## Întrebarea 216

Q:

```
10. Ce va afișa codul?  
int p[2];  
char buf[10];  
int n;  
pipe(p);  
n = read(p[0], buf, 10);  
printf("%d\n", n);
```

A:

iccametiammmntieentineti

Se va bloca deoarece pipe-urile de scriere nu au fost inchise si read va astepta date de la un scriitor inexistent.

Daca se presupune ca pipe-ul de scriere este inchis, va printa 0.

---

## Întrebarea 217

Q:

11. De ce este un proces zombie o problemă?

A:

Ocupă PID-uri și sistemul se va bloca pentru că nu vor mai fi PID-uri disponibile.

Procesele zombie sunt problematice deoarece, dacă nu sunt "așteptate" cu wait, vor crește în număr, ocupând PID-uri și aducând sistemul într-un punct în care nu mai pot fi create alte procese din cauza lipsei de PID-uri.

---

## Întrebarea 218

Q:

12. f is executed simultaneously by 10 threads. Add the necessary code to ensure n is 10 after threads have completed.

A:

```
int n=0;
```

```
void * f(void * p) {  
    net;  
    return NULL;  
}
```

---

## Întrebarea 219

Q:

13. Planificați activitățile astfel încât suma întârzierilor să fie minimă: A/7/13, B/5/9, C/2/4.  
(Format: Job/Durată/Termen)

A:

t = 0: timpul inițial

C: pornește la 0, termină la 2 (durata 2), termen 4 → întârziere 0

B: pornește la 2, termină la 7 (durata 5), termen 9 → întârziere 0

A: pornește la 7, termină la 14 (durata 7), termen 13 → întârziere 1

Ordinea optimă: CBA, întârziere totală = 1.

---

## Întrebarea 220

Q:

14. Scrieți un avantaj și un dezavantaj al cache-urilor set-asociative față de cache-urile cu acces direct.

A:

Avantaj: Cache-urile set-asociative sunt mai rapide deoarece politica de înlocuire este aplicată rar (șanse mult mai mici de cache thrashing).

Dezavantaj: Sunt de obicei mai lente și mai scumpe de construit din cauza multiplexorului de ieșire și a comparatoarelor adiționale.

În altă formulare:

Avantaj: Cache-urile set-asociative au de obicei rate mai mici de miss decât cache-urile cu acces direct de aceeași capacitate, deoarece au mai puține conflicte.

Dezavantaj: Cache-urile set-asociative sunt mai lente și mai scumpe de construit; cache-urile cu acces direct pot cauza coliziuni de cache (cache thrashing).

---

## Întrebarea 221

Q:

15. Ce pagină are cea mai mare prioritate în politica de înlocuire LRU, când se alege o pagină victimă?

**A:**

Pagina care a avut cel mai mic număr de solicitări. Dacă folosim contorul de acces: pagina victimă este cea cu cel mai mic contor. Dacă folosim matricea de referințe: pagina victimă este numărul liniei cu cele mai puține 1-uri.

LRU are o matrice  $N \times N$  de biți ( $N$  = numărul de pagini) care este umplută cu 0 și 1, iar prioritatea cea mai mare la alegerea paginii victimă o au paginile care au suma minimă pe linie.

---

## Întrebarea 222

**Q:**

16. Date 2 cache-uri set-asociative, unul cu 2 seturi de 4 pagini și unul cu 4 seturi de 2 pagini, care ar performa mai bine pentru următoarea secvență de cereri de pagini: 20, 9, 18, 27, 20, 9, 18, 27.

**A:**

Pentru cache-ul set-asociativ cu 2 seturi de 4 pagini există multe coliziuni deoarece restul împărțirii poate fi doar 0 sau 1, deci toate paginile vor cădea pe doar 2 seturi.

Cel cu 4 seturi de 2 pagini va performa mai bine, deoarece coliziunile nu se întâmplă atât de des. (Folosind formula  $K \bmod N$ , unde  $K$  = numărul paginii și  $N$  = numărul de seturi, cu mai puține seturi se cauzează cache thrashing când iterăm prin coloane.)

---

## Întrebarea 223

**Q:**

17. Câte blocuri de date pot fi referențiate prin tripla-indirectare a unui i-node, dacă un bloc conține  $N$  adrese către alte blocuri?

**A:**

$N^3$  blocuri de date.

Un bloc conține  $N$  adrese. Indirectarea triplă are 3 niveluri de indirectare:

- Pointer spre un bloc cu  $N$  adrese
- Fiecare dintre cele  $N$  adrese spre alt bloc cu  $N$  adrese
- Fiecare dintre cele  $N \times N$  adrese spre alt bloc cu  $N$  adrese
- Fiecare dintre cele  $N \times N \times N$  adrese spre un bloc de date

Total:  $N \times N \times N = N^3$  blocuri de date.

---

## Întrebarea 224

Q:

65. Considerând problema producător-consumator cu un buffer de capacitate N. Câte semafoare ați folosi pentru a asigura corectitudinea operațiilor și care ar fi valorile lor inițiale?

A:

Trei semafoare (sau 2 + un mutex):

- un semafor cu valoare inițială 0: indică numărul de poziții ocupate
- un semafor cu valoare inițială N: indică numărul de poziții libere
- un semafor binar (sau mutex) cu valoare 1: asigură accesul exclusiv la buffer când este modificat

---

## Întrebarea 225

Q:

19. Metodă pentru prevenirea deadlock-ului când nu poți evita modificarea concurentă a resurselor.

A:

choose an order for your resources (any order) and always lock them in the same order

---

## Întrebarea 226

Q:

21. Scrieți o comandă UNIX care afișează toate liniile din fișierul a.txt care conțin cel puțin un număr binar care este multiplu de 4 cu 5 sau mai multe cifre (ex: 010100).

A:

```
sa ait ath lel lle ia et te
```

```
grep -E -i "[01]{3,}[0]{2}" a.txt
```

---

## Întrebarea 227

Q:

22. Scrieți o comandă UNIX care inversează toate perechile care conțin o cifră impară urmată de o vocală (a23e8i97u3 -> a2e38i9u73).

A:

```
aZesol$u/3)).
```

```
sed -E"s/([13579])([aeiou])/\2\1/gi" a.txt
```

---

## Întrebarea 228

Q:

23. Scrieți o comandă UNIX care afișează toate scorurile unice de fotbal (ex: 4-0) care apar în a.txt. Numărul de goluri poate avea cel mult 2 cifre.

A:

goals may have at most 2 digits.

```
grep -E -i "[1-9][0-9]?-[1-9][0-9]2" a.txt | sort | uniq
```

---

## Întrebarea 229

Q:

24. Print the number of processes of every active user in the system.

A:

```
ps -ef | tail -n+2 | awk-F" " {print $1}' | sort | uniq -c
```

---

## Întrebarea 230

Q:

25. Scrieți un script UNIX Shell care calculează media fișierelor cu extensia ".txt" pe director din directorul curent și toate subdirectoarele sale.

A:

BoianPlease enter a directory

```
al $f E <i -e "*"#.txt
```

---

## Întrebarea 231

Q:

Câte procese va crea fragmentul de cod de mai jos, excluzând procesul părinte?

```
if(fork() != fork()) {  
    fork();  
}
```

A:

6 copii (7 procese inclusiv părintele).

Execuție:

- P face primul fork() → C1. În P: PID(C1), în C1: 0.

- P face al doilea fork() → C2.
- C1 face al doilea fork() → C3.
- Acum se compară valorile:
  - În P: PID(C1) != PID(C2) → true → fork() → C4
  - În C1: 0 != PID(C3) → true → fork() → C5
  - În C2: PID(C1) != 0 → true → fork() → C6
  - În C3: 0 != 0 → false → NU mai face fork

Total copii: C1, C2, C3, C4, C5, C6 = 6.

## Întrebarea 232

Q:

27. Draw the hierarchy of processes generated by the following code: \*nu va trb sa desenezi:))\*

A:

(Notă din întrebare: "nu va trb sa desenezi")

Întrebare incompletă din arhiva originală — răspunsul depinde de codul care lipsește din enunț.

## Întrebarea 233

Q:

28. Ce face apelul de sistem "write" când în PIPE există spațiu, dar nu suficient pentru cât i se cere să scrie?

A:

Scrie cât permite spațiul disponibil.

## Întrebarea 234

Q:

30. Ce se întâmplă cu procesele zombie ale unui părinte care se termină?

A:

Sunt adoptate de procesul init (PID 1), care va apela wait pentru ele, eliberând astfel resursele lor. Astfel, procesele zombie nu mai rămân ca zombi indefinit.

## Întrebarea 235

Q:

31. Planificați execuția joburilor de mai jos astfel încât suma întârzierilor să fie minimă: A/22/27, B/2/15, C/4/5.

(Format: Job/Durată/Termen)

**A:**

Ordinea: CBA — întârziere totală 1.

C: pornește la 0, termină la 4, termen 5 → întârziere 0

B: pornește la 4, termină la 6, termen 15 → întârziere 0

A: pornește la 6, termină la 28, termen 27 → întârziere 1

Întârziere totală = 1.

---

## Întrebarea 236

**Q:**

33. Ce ați adăuga la următorul fragment de cod astfel încât să afișeze "1 3 3"? Modificările nu pot elimina din execuție liniile originale de cod.

```
int n = 0;
pthread_mutex_t m[3];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_lock(&m[id]);
    n += id;
    printf("%d ", n);
    pthread_mutex_unlock(&m[(id+1) % 3]);
    return NULL;
}

int main() {
    int i;
    pthread_t t[3];
    for (i=0; i<3; i++) pthread_mutex_init(&m[i], NULL);
    for (i=0; i<3; i++) pthread_create(&t[i], NULL, f, (void*)i);
    for (i=0; i<3; i++) pthread_join(t[i], NULL);
    for (i=0; i<3; i++) pthread_mutex_destroy(&m[i]);
    return 0;
}
```

**A:**

```
PE ee eee ene ne 2 ae
return NULL;
}
int main() {
    inti;
    pthread_t t[3];
    for (i=0; i<3; i++) {
        pthread_mutex_init(&m[i], NULL);

        3;i+4) {
            pthread_create(&t[i], NULL, f, (void*)i);
        }
    }
}
```

```
for(i=0;i<3;i++) {  
pthread_join(t{i}, NULL);  
  
}  
  
for(i=0;i<3;i++) {  
  
pthread_mutex_destroy(&m[i]);  
  
}  
  
return 0;
```

---

## Întrebarea 237

Q:

38. Fișierul a.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor sale.

A:

```
awk -F" " {print $1 + $2}' a.txt
```

---

## Întrebarea 238

Q:

37. Scrieți o comandă UNIX Shell care inversează în fișierul a.txt toate perechile de cifre vecine (ex: a3972b -> a9327b).

A:

```
sed -E "s/([0-9])([0-9])\1/g" a.txt
```

```
sed 's/([[:digit:]])\1\1/g' b.txt
```

2. sed -r 's/([0-9])([0-9])\1/g'

---

## Întrebarea 239

Q:

Afișați doar liniile unui fișier a.txt care apar o singură dată (nu sunt duplicate).

A:

```
sort a.txt | uniq -u
```

Explicație:



- sort sortează liniile (necesar pentru uniq, care compară doar linii consecutive)
  - uniq -u afișează doar liniile care apar O SINGURĂ dată în fișier (-u = unique)
- 

## Întrebarea 240

Q:

40. Scrieți un script UNIX Shell care afișează numele fiecărui fișier .txt din directorul curent care conține cuvântul "cat".

A:

```
#!/bin/bash
```

```
for item in $(ls "."); do  
f="/$item"  
grep -l "cat" $f  
done
```

5.

```
#!/bin/bash
```

```
FILES=$(find -maxdepth 1 -type f -name "*.txt")
```

```
for FILE in $FILES  
do  
NR="grep -E "\<cat\>" $FILE | wc -l"  
if [ $NR -ge 1 ]  
then  
echo $FILE  
fi
```

```
done
```

---

## Întrebarea 241

Q:

41. În fragmentul de program de mai jos, marcați ce proces execută fiecare linie: Părintele, Copilul sau ambele.

```
k = fork();  
if (k == 0) {  
    printf("A\n");  
} else {  
    printf("B\n");  
}  
printf("C\n");
```

A:

```
k = fork(); Parent
```

```

if (k==0){
printf("A\n"); Child
}else {
printf("B\n"); Parent
}
printf("C\n"); Both
v6.

Pk = fork();
PC if (k e){
C printf("A\n");
}

P else {
P printf("B\n");
}

PC printf(C\n");

P k= fork();

PC if(k==0y
c printf("A\n");
}

P else {
P printf("B\n");
}

PC _ printf(C\n");

```

---

## Întrebarea 242

**Q:**

42. Câte procese vor fi create de fragmentul de cod de mai jos, excluzând procesul părinte inițial?

```
fork(); wait(0); fork(); wait(0); fork();
```

**A:**

7 procese (excluzând părintele inițial).

Execuție:

P execută: fork → C1. wait(C1). fork → C2. wait(C2). fork → C3.

C1 execută: wait (returnează -1, nu are copii). fork → C4. wait(C4). fork → C5.

C2 execută: wait (-1). fork → C6.

C3 execută: (nimic, codul s-a terminat)

C4 execută: wait (-1). fork → C7.

C5 execută: (nimic)

C6 execută: (nimic)

C7 execută: (nimic)

Total copii creați: C1, C2, C3, C4, C5, C6, C7 = 7.

---

## Întrebarea 243

**Q:**

43. Care sunt output-urile posibile în consolă ale următorului fragment de cod (ignorând orice output pe care l-ar putea genera execl), și când vor apărea?

```
printf("A\n");  
execl(...);  
printf("B\n");
```

**A:**

Dacă execl eșuează, va afișa "A" și "B".

Dacă execl reușește, va afișa doar "A" deoarece imaginea procesului curent va fi înlocuită cu una nouă.

Deci: "A\nB\n" când execl nu este executat sau eșuează; doar "A\n" în toate celelalte cazuri.

---

## Întrebarea 244

**Q:**

44. Ce face apelul de sistem "read" când pipe-ul este gol?

**A:**

Va returna EOF (valoare returnată 0) dacă niciun proces nu mai are deschis capătul de scriere. Dacă alt proces are pipe-ul deschis pentru scriere, read va bloca în anticiparea unor date noi.

---

## Întrebarea 245

**Q:**

45. Ce face apelul de sistem "open" înainte de a se întoarce când deschide un FIFO?

**A:**

Așteaptă până când capătul opus al FIFO-ului este deschis de alt proces. Apelul de sistem va aștepta până când operația complementară din celălalt proces este deschisă (deschidere pentru citire așteaptă scriere și viceversa).

---

## Întrebarea 246

Q:

46. Dați un motiv pentru a alege thread-uri în loc de procese.

A:

Comunicarea între thread-uri este mai ușoară decât între procese din cauza datelor partajate.

- Mult mai rapid de creat un thread decât un proces.
- Mult mai rapidă schimbarea între thread-uri decât între procese.
- Comunicarea inter-thread este mai rapidă decât cea inter-proces deoarece thread-urile aceluiași proces partajează memoria.

Dezavantaje:

- Un thread poate strica datele altui thread.
- Dacă un thread se blochează, toate thread-urile procesului se blochează.

Uneori e mai bine să folosești thread-uri în loc de procese deoarece thread-urile consumă mai puține resurse (au memorie partajată, folosesc variabile globale etc.), pe când procesele nu partajează date, consumă mai multe resurse și durează mai mult crearea și comunicarea între ele.

---

## Întrebarea 247

Q:

47. Considerând că funcțiile "fa" și "fb" sunt rulate în thread-uri concurente, care va fi valoarea lui "n" după ce thread-urile sunt terminate? De ce?

```
pthread_mutex_t a, b;  
int n = 0;  
  
void* fa(void* p) {  
    pthread_mutex_lock(&a);  
    n++;  
    pthread_mutex_unlock(&a);  
}  
  
void* fb(void* p) {  
    pthread_mutex_lock(&b);  
    n++;  
    pthread_mutex_unlock(&b);  
}
```

**A:**

Nu se cunoaște valoarea exactă a lui  $n$  după terminarea thread-urilor — poate fi orice între 1 și numărul de thread-uri (de fapt poate apărea race condition).

Explicație: thread-urile  $fa$  și  $fb$  folosesc mutex-uri DIFERITE ( $a$  și  $b$ ). Asta înseamnă că nu există excludere mutuală REALĂ pentru operația  $n++$ :

- Thread  $fa$  intră în mutex  $a$  și începe  $n++$  (citire  $n$ )
- Thread  $fb$  intră simultan în mutex  $b$  și face  $n++$  (citire  $n$ )
- Ambele citesc aceeași valoare, o incrementează, și o scriu înapoi → o incrementare se pierde

Mutex-urile sunt utile DOAR dacă TOATE thread-urile care accesează aceeași resursă folosesc ACELAȘI mutex. Aici protecția e degeaba.

Soluție corectă: un singur mutex pentru ambele funcții.

---

## Întrebarea 248

**Q:**

Job Duration Deadline

A 5 9

BO 7 13

Cc 12 10Schedule the following jobs so that they all meet at their deadlines A/5/9, B/7/13, C/1/10

**A:**

Ordinea ACB — toate joburile se încheie până la deadline (întârziere totală = 0).

Joburi:

- A: durată 5, deadline 9
- B: durată 7, deadline 13
- C: durată 1, deadline 10

Verificare ordine A, C, B:

- A: pornește la 0, termină la 5, deadline 9 → întârziere 0
- C: pornește la 5, termină la 6, deadline 10 → întârziere 0
- B: pornește la 6, termină la 13, deadline 13 → întârziere 0

Total întârziere: 0. Toate joburile la termen.

---

## Întrebarea 249

Q:

49. Dați un avantaj și un dezavantaj al metodei de alocare segmentată față de metoda de alocare paginată.

A:

Dezavantaj: Metoda segmentată este mai lentă decât cea paginată.

Avantaj: Metoda segmentată permite partajarea procedurilor.

Alte avantaje:

- Tabelele de segmente folosesc mai puțină memorie decât paginarea.
- Nu oferă fragmentare internă, în timp ce alocarea paginată oferă.
- Mai simplu să relocăm segmente decât întregul spațiu de adrese.

Alte dezavantaje:

- Dimensiunea inegală a segmentelor nu este bună în cazul swap-ului.
  - Necesită intervenția programatorului.
  - Greu de alocat memorie contiguă deoarece are dimensiune variabilă.
  - Algoritm costisitor de gestionare a memoriei.
- 

## Întrebarea 250

Q:

50. Când ați încărca în memorie paginile unui program care tocmai pornește?

A:

Când sunt solicitate. În momentul începerii execuției nu are nicio pagină în memorie (load la cerere / demand paging).

---

## Întrebarea 251

Q:

51. Când își schimbă un proces starea din RUN în READY?

A:

Când un alt proces care era în starea WAIT din cauza unei operații I/O a terminat operația și este adus înapoi pentru execuție (sau când cuanta de timp a procesului curent expiră). Procesul curent este pus în READY, iar primul proces din coada Ready este pus să ruleze.

---

## Întrebarea 252

Q:

52. Dat fiind un sistem de fişiere UNIX configurat cu un bloc de B octeţi care poate conţine A adrese, şi i-noduri cu S link-uri directe, un link de indirectare simplă, un link de indirectare dublă şi un link de indirectare triplă, daţi formula pentru dimensiunea maximă posibilă a unui fişier.

A:

Dimensiunea maximă =  $(S + A + A * A + A * A * A) * B$  octeţi

Unde:

- $S * B$  = octeţi accesibili prin link-urile directe
- $A * B$  = octeţi prin indirectarea simplă (un bloc cu A adrese, fiecare către un bloc de date)
- $A * A * B$  = octeţi prin indirectarea dublă
- $A * A * A * B$  = octeţi prin indirectarea triplă

---

## Întrebarea 253

Q:

67. Dat fiind un director şi toată ierarhia sa, scrieţi o comandă UNIX Shell care afişează toate subdirectoarele care conţin fişiere .c.

A:

```
find . -name "*.c" -printf "%h\n" | sort -u
```

Sau:

```
find . -type f -name "*.c" -exec dirname {} \; | sort -u
```

---

## Întrebarea 254

Q:

68. Ce va afişa fragmentul de cod de mai jos în consolă?

```
int p[2];
char buf[10];
int n;
pipe(p);
n = read(p[0], buf, 10);
printf("%d\n", n);
```

A:

Nu va afişa nimic, deoarece va rămâne blocat la apelul read. Capătul de scriere nu a fost închis, deci read va aştepta date care nu vor veni niciodată.

---

## Întrebarea 255

Q:

69. Scrieți o comandă UNIX Shell care afișează toate liniile dintr-un fișier a.txt care conțin cel puțin o distanță în kilometri.

**A:**

```
grep -E '[0-9]+\s*km\b' a.txt
```

Explicație:

- [0-9]+ = unul sau mai multe cifre
- \s\* = zero sau mai multe spații (între număr și unitate)
- km = literal "km"
- \b = margine de cuvânt (ca să nu prindă "kml" sau "kmh")

Variante: `grep -E '\b[0-9]+\s*km\b' a.txt` (cu margini de cuvânt la ambele capete)

---

## Întrebarea 256

**Q:**

70. Scrieți o comandă UNIX Shell care afișează liniile fișierului a.txt înlocuind orice perechi de vocală majusculă urmată de cifră pară (ex: a23E712BU4 -> a23E721B4U).

**A:**

```
sed -E 's/([AEIOU])([02468])\2\1/g' a.txt
```

---

## Întrebarea 257

**Q:**

71. Scrieți o comandă UNIX Shell care afișează o listă a tuturor aparițiilor unice ale numelor de utilizator SCS aparținând secțiunilor engleză sau română care apar într-un fișier text a.txt.

**A:**

```
grep -oE '\b[a-z]+(_[eler])[0-9]+\b' a.txt | sort -u
```

(Presupunând că username-urile au sufix \_e, \_l, \_r etc. pentru secțiuni.)

---

## Întrebarea 258

**Q:**

76. Ce face apelul de sistem "write" când FIFO-ul conține mai puțin spațiu decât datele pe care i se cere să le scrie, dar nu este plin?

**A:**

Va scrie cât de multe date poate (cât permite spațiul disponibil).



---

## Întrebarea 259

Q:

77. Dați operatorii pentru următoarele verificări Shell:

- șir nu e gol:
- este executabil:
- este diferit (pentru numere):
- este director:

A:

- șir nu e gol: `-n` (sau echivalent: `[ "$x" != "" ]`)
- este executabil: `-x`
- este diferit (numere): `-ne`
- este director: `-d`

---

## Întrebarea 260

Q:

1. Scrieți o comandă UNIX Shell care afișează liniile dintr-un fișier care conțin cuvinte ce încep cu literă mare.

A:

`grep -E '\b[A-Z][a-z]*\b' a.txt`

---

## Întrebarea 261

Q:

3. Fișierul c.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor sale.

A:

`awk '{print $1 + $2}' c.txt`

Explicație:

- awk procesează fiecare linie automat
- `$1` = prima coloană, `$2` = a doua coloană
- `print $1 + $2` = afișează suma lor
- Implicit AWK consideră spațiul/tabul ca delimitator de coloane

---

## Întrebarea 262

Q:

4. Display only the lines of a file d.txt that appear only once.

**A:**

`sort d.txt | uniq -u`

4. `sort d.txt | uniq -u`

---

## Întrebarea 263

**Q:**

16. Când își schimbă un proces starea din RUN în READY?

**A:**

Când cuanta de timp a procesului curent expiră sau când task-ul se completează sau când e îndeplinită altă condiție, procesul curent este pus în coada Ready (dacă mai are instrucțiuni de executat) sau este marcat ca finalizat (dacă a terminat). În oricare caz, primul proces din coada Ready este scos și pus să ruleze.

---

## Întrebarea 264

**Q:**

7. Câte procese sunt create de fragmentul de cod de mai jos, când procesul părinte P apelează `f(3)`?

```
void f(int n) {
    if (n > 0 || fork() == 0) {
        f(n-1);
        exit(0);
    }
    wait(0);
}
```

**A:**

Programul nu se termină – creează un număr infinit de procese (sau limitat doar de resursele sistemului).

Analiza:

- `f(n)` cu `n > 0`: condiția "`n > 0 || fork() == 0`" este true (datorită lui `n > 0`, `fork` nu se mai apelează – short-circuit evaluation). Apelează `f(n-1)`, apoi `exit`.
- `f(0)`: condiția "`0 > 0 || fork() == 0`" se evaluează la `fork() == 0`.
  - În părinte: `fork()` returnează `PID > 0` → false → `wait(0)`
  - În copil: `fork()` returnează `0` → true → apelează `f(-1)`
- `f(-1)`: condiția "`-1 > 0 || fork() == 0`" se evaluează la `fork() == 0`.
  - În părinte: face `wait`
  - În copil: face `f(-2)` → ... INFINIT

Fiecare proces nou creat continuă să facă `fork` și să apeleze `f(n-1)`, creând un lanț infinit.

---

## Întrebarea 265

**Q:**

5. Considerați că funcția `f` este executată simultan de 10 thread-uri. Ce ați adăuga pentru a asigura că valoarea lui `n` este 10 după ce thread-urile s-au terminat? În acest context, cum se numește linia `"n++"` și cum se numește variabila `n`?

```
pthread_mutex_t m;  
int n = 0;  
  
void* f(void* p) {  
    pthread_mutex_lock(&m);  
    n++;  
    pthread_mutex_unlock(&m);  
    return NULL;  
}
```

**A:**

Codul deja are mutex, deci valoarea lui `n` va fi 10 dacă există 10 thread-uri care apelează `f`.

- Linia `"n++"` este într-o secțiune critică.
  - Variabila `n` este o resursă partajată (shared variable).
-